

# MapHarvest

Every runtime flow, module and decision point of the Google Maps scraping engine — at code level, with a verified defect audit.

|                           |                                                                                                                                                                                                                                              |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SCOPE                     | every runtime flow, every module, every decision point, at code level.                                                                                                                                                                       |
| REPO STATE AUDITED        | branch <code>main</code> , HEAD <code>3399536</code> , 19 Python files / 4,035 LOC.                                                                                                                                                          |
| ENVIRONMENT AUDITED       | Windows 11 Pro, <code>venv/</code> = <b>Python 3.14.4</b> , PyQt5 + Selenium +undetected-chromedriver + lxml.                                                                                                                                |
| TEST STATUS AT AUDIT TIME | <code>tests/test_parse.py</code> PASS (8 cards, 2 sponsored), <code>tests/test_enrich_filters.py</code> PASS (all 5 cases) — but see <a href="#">A11</a> : the parser fixture is untracked, so the feed assertions only run on this machine. |
| AUDIT RESULT              | 20 findings (4 high, 7 medium, 9 low), each adversarially verified; 7 candidates refuted.                                                                                                                                                    |

## Table of contents

---

1. System in one page
2. Runtime topology — processes, threads, network
3. Module map and dependency rules
4. The record: the single data structure everything moves
5. FLOW 1 — Cold start / boot
6. FLOW 2 — Input screen: build, persist, validate, launch
7. FLOW 3 — Settings persistence
8. FLOW 4 — Worker startup and Chrome acquisition
9. FLOW 5 — Task matrix (domain × area) and budget arithmetic
10. FLOW 6 — Search navigation, consent, block detection
11. FLOW 7 — Feed discovery and scroll pagination
12. FLOW 8 — Feed HTML → card dict (the extraction core)
13. FLOW 9 — The two-gate filter pipeline
14. FLOW 10 — Work planning: what forces a detail visit
15. FLOW 11 — Detail-page pass
16. FLOW 12 — Website enrichment (email + socials)
17. FLOW 13 — Emit: worker thread → GUI thread → table
18. FLOW 14 — The five ways a task terminates
19. FLOW 15 — Multi-task handshake
20. FLOW 16 — CSV export
21. FLOW 17 — Pause, Stop, Abort, Close (the shutdown ladder)
22. FLOW 18 — Failure paths and the debug dump
23. FLOW 19 — Build & packaging pipeline
24. Performance model: where the seconds go
25. Selector registry (the fragile surface)
26. Test inventory
27. System audit — findings
28. Extension points

---

## 1. System in one page

---

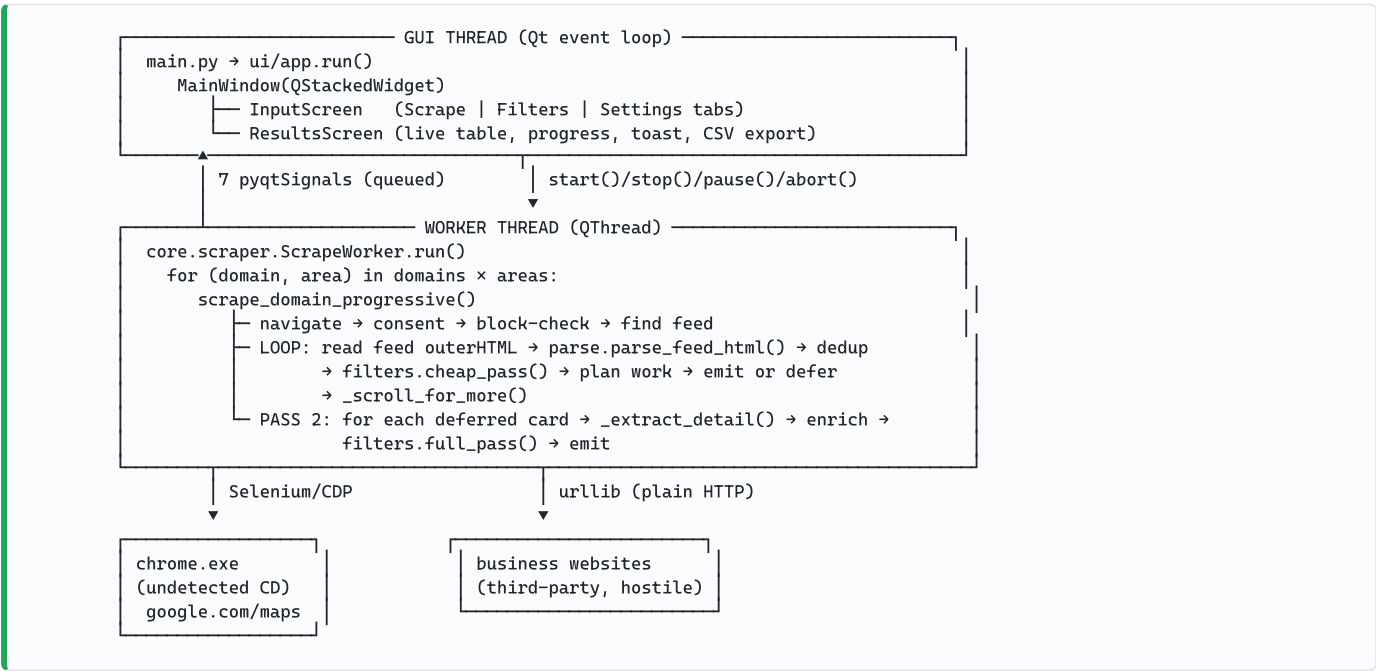
MapHarvest is a **single-process Windows desktop app** that turns a Google Maps text search ( "`<domain> in <area>`" ) into a CSV of business records.

The architectural bet — stated in `core/scrapper.py:1-14` and `core/parse.py:1-12` — is **card-first, detail-on-demand**:

The Google Maps results feed HTML *already contains* name, rating, category, address, phone and website, and each place URL encodes latitude, longitude and a stable Place ID. Parse that once per scroll and you never open a browser tab per business.

Everything else in the codebase is a consequence of that bet:

| CONSEQUENCE                                                                                        | WHERE IT LIVES                                                         |
|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| A pure, offline-testable HTML→dict parser                                                          | <code>core/parse.py</code>                                             |
| Browser automation reduced to <i>navigate</i> + <i>scroll</i> + <i>read</i> <code>outerHTML</code> | <code>core/scrapper.py:553-731</code>                                  |
| A "do I actually need to open this listing?" planner                                               | <code>core/scrapper.py:524-543</code>                                  |
| A two-gate filter system split by data cost                                                        | <code>core/filters.py</code>                                           |
| An HTTP-only (no browser) enrichment path                                                          | <code>core/enrich.py</code>                                            |
| Streaming results into the UI as they are decided                                                  | <code>pyqtSignal</code> fan-out, <code>core/scrapper.py:736-743</code> |



**Two external dependencies you cannot control:** Google Maps' DOM/class names, and arbitrary business websites. Every design choice about fallbacks and `try/except` in this codebase traces back to one of those two.

## 2. Runtime topology — processes, threads, network

### Processes

| PROCESS                                                     | CREATED BY                                                      | LIFETIME                                                                                                                 |
|-------------------------------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>python.exe</code> / <code>MapHarvest-x.y.z.exe</code> | user                                                            | app session                                                                                                              |
| <code>chrome.exe</code> (+ renderer children)               | <code>uc.Chrome(...)</code> at <code>core/scrapper.py:94</code> | one per scrape run — created in <code>ScrapeWorker.run()</code> (:814), destroyed in its <code>finally</code> (:869-874) |
| <code>chromedriver.exe</code>                               | undetected-chromedriver, <code>use_subprocess=True</code> (:90) | tied to the Chrome instance                                                                                              |

Critically: **one Chrome instance serves the entire run**, across every (domain, area) task and every detail-page visit. There is no tab pooling and no second driver. `use_tabs` still exists as a constructor parameter (`core/scrapper.py:753`) but is explicitly dead — a leftover of the pre-rewrite "tab per business" design.

Threads

| THREAD                              | OWNS                                                                      | MUST NEVER TOUCH                                                               |
|-------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| GUI thread                          | every <code>QWidget</code> , the table, timers, <code>QApplication</code> | the Selenium driver (except <code>abort()</code> , deliberately — see FLOW 17) |
| <code>ScrapeWorker</code> (QThread) | the driver, the record dicts, all parsing/filtering                       | any widget — it only emits signals                                             |

Cross-thread communication is exclusively the seven signals declared at `core/scrapper.py:736-743`:

```
log_signal          = pyqtSignal(str, str)           # message, status("active"|"done")
progress_signal     = pyqtSignal(int)                # running collected count
result_signal       = pyqtSignal(dict)               # one finished record
domain_finished_signal= pyqtSignal(str, str, int, int, bool) # domain, area, count, max, hit_limit
done_signal         = pyqtSignal()                   # run over (always fires - finally block)
error_signal        = pyqtSignal(str)                 # fatal exception text
paused_signal       = pyqtSignal(bool)
```

Because both objects live in the same process and the receiver lives in the GUI thread, Qt uses **queued connections** — the emitted `dict` is handed over by reference and delivered on the GUI thread's next event-loop turn.

Network

| CHANNEL                     | CODE                                                       | NOTES                                                                                                                  |
|-----------------------------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Google Maps, browser-driven | <code>driver.get()</code> at <code>:559, :453, :687</code> | forced <code>hl=en (:103)</code> + <code>intl.accept_languages=en-US (:86)</code> so status strings stay parseable     |
| Business websites, raw HTTP | <code>core/enrich.py:59-74</code>                          | <code>urllib.request</code> only, spoofed Chrome UA ( <code>:21-24</code> ), 8s timeout, 1.5 MB read cap, gzip handled |

No API keys, no accounts, no server component, no telemetry. All state is local.

Disk

| PATH                                                               | WRITTEN BY                            | CONTENTS                                                                     |
|--------------------------------------------------------------------|---------------------------------------|------------------------------------------------------------------------------|
| <code>~/.mapharvest/settings.json</code>                           | <code>core/settings.py:40-46</code>   | headless flag, slider cap, default max, export dir, ≤12 saved searches       |
| <code>&lt;export_dir&gt;/&lt;domain&gt;_in_&lt;area&gt;.csv</code> | <code>core/exporter.py:49-57</code>   | UTF-8 <b>with BOM</b> ( <code>utf-8-sig</code> ) so Excel opens it correctly |
| <code>./debug/&lt;tag&gt;.png</code> + <code>.html</code>          | <code>core/scrapper.py:512-520</code> | forensic dump on the three "nothing found" paths                             |

### 3. Module map and dependency rules



Enforced dependency rules, worth preserving:

- 1. `core/` never imports `ui/`. One-way dependency; the engine is headless-testable.
- 2. `parse.py` and `filters.py` are pure. No Selenium, no network, no clock. That is what makes `tests/` able to run offline against a captured fixture.
- 3. `enrich.py` uses `stdlib` only (`urllib`, `gzip`, `re`) — deliberate, so enrichment adds no dependency to `requirements.txt` and no extra `PyInstaller` hook.
- 4. `distutils_compat` must be imported before `undetected_chromedriver` — `core/scrapers.py:23` does it with a `*` import specifically so the side effect (`_install_shim()` at `core/distutils_compat.py:89`) runs first. Reordering those imports breaks the app on Python 3.12+.

#### Why `distutils_compat` exists

Python 3.12 removed `distutils`. `undetected-chromedriver` still does `from distutils.version import LooseVersion` and then reads `release.version[0]` to get Chrome's major version. `core/distutils_compat.py:16-71` reimplements `LooseVersion` — including the `.version` list attribute and the six comparison operators — and `:74-89` injects synthetic `distutils / distutils.version` modules into `sys.modules`. The guard at `:76-78` refuses to overwrite a *real* `distutils` that already provides a working `LooseVersion`.

### 4. The record: the single data structure everything moves

One flat `dict` per business is created by the parser, mutated in place by the detail and enrich passes, then emitted. There is no class, no schema validation — the contract is the key set.

#### Public keys (exportable columns)

| KEY                   | ORIGIN                                                                                                                        | UPGRADEABLE BY<br>DETAIL VISIT?            | CSV LABEL<br>( <code>CORE/EXPORTER.PY:4-28</code> ) |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|-----------------------------------------------------|
| <code>name</code>     | card anchor <code>aria-label</code> ( <code>parse.py:132</code> ), fallback <code>div.qBF1Pd</code> ( <code>:137-139</code> ) | no                                         | Business Name                                       |
| <code>category</code> | first token of an info row ( <code>parse.py:227</code> )                                                                      | yes<br>( <code>scraper.py:479-480</code> ) | Category                                            |

| KEY                                             | ORIGIN                                                                     | UPGRADEABLE BY<br>DETAIL VISIT?          | CSV LABEL<br>( CORE/EXPORTER.PY:4-28 ) |
|-------------------------------------------------|----------------------------------------------------------------------------|------------------------------------------|----------------------------------------|
| rating                                          | span.MW4etd<br>( parse.py:150 ), fallback<br>star aria-label ( :156-159 )  | yes<br>( scraper.py:481-486 )            | Rating                                 |
| review_count                                    | span.UY7F9 / RDApEe ,<br>fallback aria-label<br>( parse.py:187-200 )       | yes                                      | Review Count                           |
| address                                         | last token of an info row<br>( parse.py:228 )                              | yes<br>( scraper.py:459-464 )            | Address                                |
| website                                         | a[data-value="Website"]<br>minus ad redirects<br>( parse.py:170-173 )      | fills if empty<br>( scraper.py:475-478 ) | Website                                |
| phone                                           | span.UsdLk<br>( parse.py:166 ), fallback<br>regex over rows ( :236-244 )   | fills if empty<br>( scraper.py:465-474 ) | Phone                                  |
| maps_link                                       | href.split("?")[0]<br>( parse.py:47 )                                      | no                                       | Maps Link                              |
| latitude / longitude                            | !3d...!4d... in the URL,<br>fallback @lat,lng<br>( parse.py:28-31, 49-53 ) | no                                       | Latitude / Longitude                   |
| place_id                                        | !19s... in the URL<br>( parse.py:29, 55-57 )                               | no                                       | Place ID                               |
| hours                                           | <b>detail only</b> —<br>_extract_hours<br>( scraper.py:367-387 )           | n/a                                      | Hours                                  |
| review_1..3                                     | <b>detail only</b> —<br>_extract_reviews<br>( scraper.py:404-425 )         | n/a                                      | Review 1..3                            |
| email                                           | <b>enrich only</b><br>( enrich.py:105-130 )                                | n/a                                      | Email                                  |
| facebook, instagram, linkedin, twitter, youtube | <b>enrich only</b><br>( enrich.py:132-143 )                                | n/a                                      | Facebook ... YouTube                   |
| domain, area                                    | injected per task<br>( scraper.py:507-509 )                                | n/a                                      | Search Domain / Search Area            |

## Bookkeeping keys (never exported)

| KEY            | SET AT             | PURPOSE                                                                |
|----------------|--------------------|------------------------------------------------------------------------|
| _href          | parse.py:131       | raw place URL, used for the detail visit and the "open in Maps" action |
| _sponsored     | parse.py:144-147   | ad card → skipped at scraper.py:637                                    |
| _key           | parse.py:141       | dedup identity: place_id → cid → URL slug → name ( parse.py:66-76 )    |
| _domain, _area | scraper.py:503-504 | always present, even when the user didn't tick those columns           |

| KEY                 | SET AT                      | PURPOSE                                                       |
|---------------------|-----------------------------|---------------------------------------------------------------|
| <code>_error</code> | <code>scraper.py:495</code> | detail-extraction exception text (see <a href="#">audit</a> ) |

### Type invariant

Every value is a string; missing means `""`, never `None`. `parse.py:107-122` initialises all keys to `""`, and `_extract_detail` returns only truthy values (`scraper.py:496`). Downstream code relies on this: `filters._to_float / _to_int` (`filters.py:18-30`) coerce and default to 0, and the table renderer substitutes an em-dash for empty (`screen_results.py:366`).

### Identity and dedup

place\_key(href)

!19s<ChIJ...>

!1s<0x...:0x...>

/place/<slug>

href without query

core/parse.py:66-76

→ Place ID

→ CID hex pair

→ URL-decoded slug

→ last resort

(stable, preferred)

(stable fallback)

(weak)

Dedup is a `set` of `_key` values scoped to **one (domain, area) task** (`scraper.py:597`, checked at `:640-643`). Consequences: the same business appearing in two cities' searches yields two records in two different CSVs, and the set resets per task.

---

## FLOW 1 — Cold start / boot

```
python main.py
main.py:1      from ui.app import run
main.py:4      run()
app.py:526     QApplication(sys.argv)
app.py:527     setStyle("Fusion")           ← neutral base so the QSS is the only theme
app.py:528     load_font(app)               ← "DM Sans" if installed, else Segoe UI / AppleSystemUIFont
app.py:529     setStyleSheet(QSS)          ← the 430-line dark theme, app.py:12-441
app.py:530     _install_excepthook()       ← a raising slot prints a traceback instead of killing the GUI
app.py:532     MainWindow()
:457           title "MapHarvest"
:458           setFixedSize(820, 640)     ← input-screen geometry
:460-467       QStackedWidget[ InputScreen, ResultsScreen ]
:469-471       wire start / stop / home signals
app.py:542     signal.signal(SIGINT, _on_sigint)
app.py:543-545 QTimer(200 ms, no-op)
app.py:546     aboutToQuit → window.shutdown_worker
app.py:548     window.show()
app.py:549     sys.exit(app.exec_())
```

Two boot details that look like noise and are not:

- **The idle `QTimer` at `app.py:543-545`**. Qt's event loop is C++; while it blocks, no Python bytecode runs, so a `SIGINT` sits queued until some slot happens to execute — historically surfacing as a bogus `KeyboardInterrupt` traceback inside whatever button the user clicked next. The 200 ms no-op timer gives the interpreter a scheduling slot. The comment at `:534-537` records this.
- **`setStyle("Fusion")` before the stylesheet**: on Windows the native style ignores several QSS properties (notably `QCheckBox::indicator` and header padding), which would make the dark theme render half-applied.

`InputScreen.__init__` (`screen_input.py:43-50`) loads settings **before** building widgets, then calls `_apply_settings_to_ui()` (`:477-493`) after — so the widget tree is created with defaults and then reconciled with the persisted state, with `blockSignals` guards (`:480-482`, `:484-486`, `:488-490`) preventing the restore from firing the "user changed it" handlers.



## FLOW 2 — Input screen: build, persist, validate, launch

### 2.1 The three tabs

`_build()` (`screen_input.py:52-89`) puts a `QButtonGroup` of three checkable buttons over a `QStackedWidget`; `idClicked` → `setCurrentIndex` (`:89`) is the entire tab mechanism.

| TAB      | BUILDER               | PRODUCES                                                  |
|----------|-----------------------|-----------------------------------------------------------|
| Scrape   | <code>:92-273</code>  | domains, areas, max-results, export dir, field checkboxes |
| Filters  | <code>:276-396</code> | the filter spec dict                                      |
| Settings | <code>:399-445</code> | headless flag, slider cap                                 |

### 2.2 Field checkboxes and the speed classes

`FIELD_KEYS` (`:21-27`) and `FIELD_NAMES` (`:28-34`) are positionally paired and zipped at `:217` into a 2-column grid. The important logic is the default state:

```
DETAIL_FIELDS      = {"hours", "review_1", "review_2", "review_3"}    # :37 → opens each listing
ENRICH_FIELDS      = {"email", "facebook", "instagram", "linkedin", "twitter", "youtube"} # :39 → fetches each site
DEFAULT_OFF_FIELDS = DETAIL_FIELDS | ENRICH_FIELDS                    # :41
cb.setChecked(key not in self.DEFAULT_OFF_FIELDS)                    # :219
```

So the 15 card-derived fields are on by default and the 10 expensive ones are off, with per-checkbox tooltips explaining the cost (`:220-223`). `DETAIL_FIELDS` here mirrors `scraper.DETAIL_ONLY_FIELDS` (`scraper.py:42`) and `ENRICH_FIELDS` mirrors `enrich.ENRICH_KEYS` (`enrich.py:56`) — **three copies of the same knowledge in two layers**; they must be edited together.

The "All / Fast only / None" buttons (`:239-247`) map to `_select_all_fields`, `_select_fast_fields` (which re-applies exactly the default rule) and `_select_no_fields` (`:453-463`).

### 2.3 Multi-domain and multi-area lists

`ListDialog` (`domain_list_dialog.py:7-66`) is a modal `QTextEdit`: one item per line, `_save()` (`:60-63`) strips and drops blanks. `DomainListDialog` (`:69-83`) is a thin preset of it, kept for name compatibility.

Merging happens in `_get_domains` / `_get_areas` (`screen_input.py:597-613`): the main input goes first, then the list items, deduped **case-insensitively while preserving the original casing** — the `seen` set holds `.lower()` but the appended value is the raw string.

### 2.4 Validation gate

`validate()` (`:639-662`) is a fail-fast chain returning `None` on failure, with a *specific* visual response per failure:

| CONDITION                                                                           | RESPONSE                                                   |
|-------------------------------------------------------------------------------------|------------------------------------------------------------|
| no domains                                                                          | jump to Scrape tab, <code>shake(domain_input)</code>       |
| no areas                                                                            | jump to Scrape tab, <code>shake(area_input)</code>         |
| export dir empty <b>or not an existing directory</b> ( <code>os.path.isdir</code> ) | jump + shake <code>export_dir_input</code>                 |
| no fields ticked                                                                    | jump + <code>_flash_fields_label()</code> (red for 600 ms) |

`shake()` (:668–679) animates the widget's `pos` property through  $\pm 6$  px over 200 ms and stores the animation on `self._anim` — necessary, because a `QPropertyAnimation` that goes out of scope is garbage-collected mid-animation. Note the single-slot storage: a second shake replaces the first.

An export directory is **mandatory**, not optional — CSV writing is automatic per task (see [FLOW 16](#)).

## 2.5 Launch

```
_on_start()                                screen_input.py:685–698
    validate() → (domains, areas, fields, export_dir)
    add_saved_search(settings, domains, areas[0], limit) ← only the FIRST area is persisted
    _refresh_saved_list(); _persist_settings()
    start_signal.emit(domains, areas, fields, headless, limit, export_dir, filters)
    ↓
MainWindow.on_start()                      app.py:473–480
    results_screen.setup(...) ← reset table/state, build columns
    setFixedSize(1040, 740) ← the window grows for the results view
    stack.setCurrentIndex(1)
    results_screen.start_worker() ← construct + start the QThread
```

`get_filters()` (:618–631) snapshots the Filters tab into the plain dict that `core/filters.normalize_spec` expects; the website tri-state combo is encoded as two booleans (`require_website` = index 1, `require_no_website` = index 2, :624–625).

---

## FLOW 3 — Settings persistence

```
~/mapharvest/settings.json      core/settings.py:14-15
{
  "headless": false,             # Settings tab checkbox
  "max_limit_cap": 100,          # Settings tab spinbox → slider maximum
  "default_max_results": 50,     # last slider value
  "export_dir": "",             # last chosen folder
  "saved_searches": [ {domains, area, max_results}, ... ] # ≤ MAX_SAVED_SEARCHES = 12
}
```

- **Load** (:22-37) — `_ensure_dir()`, return defaults if the file is absent, else `json.load` and merge **only keys that exist in `DEFAULT_SETTINGS`** (:30), so unknown keys from a future/hand-edited file are dropped rather than propagated. `saved_searches` is type-checked and truncated (:31-34). Any exception → pristine defaults (:36-37); a corrupt file therefore degrades silently instead of blocking startup.
- **Save** (:40-46) — builds the payload from `DEFAULT_SETTINGS` keys only (:43), so the written file is always exactly the known schema. Not atomic: a crash mid-write can truncate the file (recovered on next load by the `except` path).
- **add\_saved\_search** (:49-59) — MRU semantics: remove an identical entry, insert at the front, truncate to 12, save immediately, return the mutated settings dict.

Write triggers: headless toggle (`screen_input.py:414`), slider-cap change (:434), export-folder choice (:517-523), and Start (:694). The slider value itself is only mirrored into the in-memory dict on change (:506-508) and persisted at the next `_persist_settings()` call.

Restore of a saved search is `_load_saved_search` (:545-559): first domain into the line edit, the rest into `_extra_domains`, the area, and the limit clamped to the current slider maximum. Fields and filters are deliberately **not** part of a saved search.

## FLOW 4 — Worker startup and Chrome acquisition

### 4.1 Overlap guard (GUI side)

`ResultsScreen.start_worker()` (`screen_results.py:275-312`) does three things before creating anything:

1. **Refuses to overlap** (`:278-282`): if a previous worker is still running, `stop()`, wait 5 s, then `abort()`, wait 5 s. Without this, a second `chrome.exe` would run alongside the first.
2. **Disconnects the old worker's seven signals** (`:284-297`), each in its own `try/except TypeError`, because disconnecting a never-connected signal raises.
3. Constructs a fresh `ScrapeWorker`, reconnects (`:304-311` — `domain_finished_signal` **only in multi mode**, `:310-311`), and `start()`s the thread.

### 4.2 Chrome version detection

`_chrome_major_version()` (`scraper.py:50-73`) resolves the installed Chrome major so undetected-chromedriver can be pinned with `version_main`:

```
1. scan %ProgramFiles%\Google\Chrome\Application
   %ProgramFiles(x86)%\...
   %LocalAppData%\... for a directory named \d+\.d+\.d+\.d+
2. else read HKCU\Software\Google\Chrome\BLBeacon\version
3. else return 0 → version_main omitted, uc auto-detects
```

### 4.3 Driver options

`get_driver(headless)` (`:76-97`):

| OPTION                                                                                                                                               | LINE                                   | WHY                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>--headless=new</code> (conditional)                                                                                                            | <code>:79</code>                       | modern headless; the old mode is trivially fingerprinted                                                                                                          |
| <code>--disable-notifications</code> , <code>--no-first-run</code> , <code>--no-default-browser-check</code> , <code>--disable-popup-blocking</code> | <code>:80-84</code>                    | remove interstitials that would sit on top of the feed                                                                                                            |
| <code>--lang=en-US</code> + <code>prefs intl.accept_languages</code>                                                                                 | <code>:82</code> ,<br><code>:86</code> | <b>load-bearing:</b> <code>_is_end_of_list</code> ( <code>:199-210</code> ) and <code>_dismiss_consent_screen</code> ( <code>:107-131</code> ) match English text |
| <code>page_load_strategy = "eager"</code>                                                                                                            | <code>:87</code>                       | return at DOMContentLoaded — Maps never reaches full <code>load</code>                                                                                            |
| <code>use_subprocess=True</code>                                                                                                                     | <code>:90</code>                       | uc's own requirement for reliable patched-driver launch                                                                                                           |
| <code>set_window_size(1280, 900)</code>                                                                                                              | <code>:95</code>                       | a viewport that renders the desktop feed layout, not the mobile one                                                                                               |
| <code>set_page_load_timeout(30)</code>                                                                                                               | <code>:96</code>                       | bounds every <code>driver.get()</code> so a hung navigation can't wedge the thread forever                                                                        |

Failure here (no Chrome, driver/browser version mismatch, AV interference) raises inside `run()` and lands in the single `except` at `:866-867` → `error_signal` → `ResultsScreen.on_error` (`:564-566`) shows `Error: ...` and flips the UI to idle. The `finally` (`:869-876`) still runs, so `done_signal` always fires exactly once per run.

## FLOW 5 — Task matrix (domain × area) and budget arithmetic

`Scrapeworker.run()` (`scraper.py:811-876`):

```
tasks = [(d, a) for d in self.domains for a in self.areas] # :817 full cross product
multi = len(tasks) > 1                                   # :818 the mode switch
```

`multi` changes four behaviours:

| BEHAVIOUR          | MULTI = FALSE (ONE TASK)                                                                                          | MULTI = TRUE (≥2 TASKS)                                                |
|--------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Result limit       | <code>max_results</code> is a <b>global</b> budget: <code>limit = max_results - total_collected</code> (:826-829) | <code>max_results</code> is a <b>per-task</b> budget (:825)            |
| Progress base      | <code>_progress_base = total_collected</code> (:833) — the counter keeps climbing                                 | <code>0</code> — the counter restarts each task                        |
| Per-task log lines | suppressed (:844-847)                                                                                             | emitted (:849-858)                                                     |
| Handshake          | none                                                                                                              | <code>domain_finished_signal</code> + wait for the UI's ack (:860-864) |

The UI computes the same mode independently: `_multi = len(domains) * len(areas) > 1` (`screen_results.py:241`), and prepends the `area` / `domain` pseudo-columns when more than one of each was requested (:244-247) so the CSV can tell rows apart.

Inside a task, the budget becomes a *collection* budget (`scraper.py:590-595`):

```
target      = max_results if max_results > 0 else 10**9      # 0 = unlimited
strict      = needs_phone(spec) or needs_email(spec) or needs_reviews(spec)
collect_cap = target if not strict else min(target*4, target + 300)
```

**Why over-collect:** a "must have a phone/email" or review-count filter can only be decided *after* a detail visit or a website fetch, so some candidates will be rejected late. `collect_cap` lets the loop gather up to 4× the target (capped at +300) of *candidates*, so the target can still be met after late rejections. When no late-decided filter is active, `collect_cap == target` and the loop stops the moment the target is reachable.

## FLOW 6 — Search navigation, consent, block detection

```
_search_url(domain, area)                                scraper.py:100-103
"https://www.google.com/maps/search/" + quote(f"{domain} in {area}") + "?hl=en"

driver.get(url); sleep(1.0)                               :559-560
↓
_dismiss_consent_screen(driver)                           :107-131
  try 5 XPaths in order: "Accept all" (div-in-button), "Accept all", "Reject all",
  "I agree", form button[aria-label*=Accept]
  → first visible match is clicked via execute_script (never .click(); overlays intercept
    native clicks), sleep 0.8, return True
    ↓
    if clicked
    ↓
    log "Dismissed cookie-consent screen..."; sleep 1.0    :563-565
    still on consent.google.com? → re-GET the search URL, dismiss again, sleep :566-570
    ↓
    _page_blocked_reason(driver)                          :134-148
    consent.google.com in URL → "Stuck on Google's cookie-consent page..."
    "sorry/index" in URL or "unusual traffic" in the first 20 KB of source
    → "unusual traffic / CAPTCHA page..."
    "recaptcha" in source and not a maps URL → "reCAPTCHA challenge instead of Maps"
    else ""
    ↓
    if reason
    ↓
    _debug_dump("blocked_<domain>_<area>"); log the reason; return (0, True) :573-576
    ↓
    else
    ↓
    WebDriverWait(12s) until _get_feed(d) is not None      :578-579
    timeout → recheck block reason, dump "nofeed_*", log, return (0, True) :580-588
```

Note the return convention: `(matched, completed)` — these early exits return `completed=True`, meaning "this task is finished, don't treat it as interrupted". That flag drives the log wording at `:849-858`, not any retry.

`_page_blocked_reason` reads only the first 20,000 characters of `page_source` (`:138`) — enough to catch Google's interstitials while keeping the string cheap on a full Maps document.

## FLOW 7 — Feed discovery and scroll pagination

### 7.1 Finding the feed

`_get_feed` (:152–174) tries three CSS selectors, requiring `is_displayed()`, then falls back to a 6-second `WebDriverWait` on the XPath union of the same three:

```
div[role="feed"] → div[aria-label*="Results for"] → div[aria-label*="Search results"]
```

`_feed_html` (:213–223) returns `feed.get_attribute("outerHTML")`, and explicitly re-resolves the element once on `StaleElementReferenceException` (:219–221) — Maps replaces the container during lazy loads, so a cached handle goes stale mid-run.

### 7.2 The scroll fix (the single most important browser interaction)

```
// _JS_FEED, scraper.py:228–232 – resolve the container in-page, same fallbacks
function feedEl(){ return document.querySelector('div[role="feed"]')
                    || document.querySelector('div[aria-label^="Results for"]')
                    || document.querySelector('div[aria-label*="Search results"]'); }

// _nudge_feed, scraper.py:246–268
const f = feedEl();
f.scrollTop = f.scrollHeight;
f.dispatchEvent(new WheelEvent('wheel', {deltaY: 1500, bubbles: true, cancelable: true}));
```

Three failures this specific implementation encodes (comments at :248–254):

1. **`scrollTop` alone does nothing.** Maps listens for wheel events, not scroll position. Setting `scrollTop` without dispatching the `WheelEvent` stalls pagination at the first page.
2. **`scrollIntoView()` is worse** — it fights the container's own scroll management and stalls.
3. **"the div with the most links" heuristics break.** Once results load, such a heuristic latches onto an inner wrapper that never scrolls, and pagination freezes. Hence the explicit `role="feed"`-first resolution.

`_scroll_for_more` (:271–286) is a growth-detecting loop, not a fixed sleep:

```
before = _feed_card_count(driver)          # counts a[href*="/maps/place"] inside the feed, :235–243
_nudge_feed()
loop for ≤ timeout (default 5 s):
    sleep 0.4
    count > before          → return True      (feed grew)
    _is_end_of_list()      → return False     (Google says that's all)
    _nudge_feed()          ← keep nudging; one wheel event is often ignored
return count > before
```

`_is_end_of_list` (:199–210) is guarded by `_is_loading` (:177–196) so an in-flight fetch is never mistaken for the end. `_is_loading` is defensive about Maps' phantom spinners: it ignores elements that are not `is_displayed()`, that carry `opacity: 0`, or that have zero width/height — all three states appear in Maps' DOM while nothing is actually loading.

Historical note recorded in `TODO.md:15`: before this fix, runs capped at roughly 7 results.

## FLOW 8 — Feed HTML → card dict (the extraction core)

`parse_feed_html(html)` (`parse.py:247-261`):

```
lxml.html.fromstring(html)
cards = //div[class token "Nv2PK"] :252
    fallback: //div[@role="article"][./a[contains(@href,"/maps/place")]] :255
for each card: parse_card(c); keep it if it has _href or name :257-260
```

Class matching uses `_has_class` (`parse.py:88-92`), an XPath predicate on `concat(" ", normalize-space(@class), " ")` — a **whitespace-delimited token** match. Plain `contains(@class, "W4Efsd")` would also match `W4EfsdSomethingElse`; Google's class soup makes that a real hazard.

### 8.1 `parse_card` field by field (`parse.py:101-184`)

**Anchor** → **name + URL** (`:124-134`) `./a[class token hfpxzc]`, fallback any `a[href*="/maps/place"]`. The business name is the anchor's `aria-label` — more reliable than the visible headline, which truncates. Fallback name: `div.qBF1Pd` (`:136-139`).

**URL decoding** (`parse_place_url`, `:41-63`)

```
/maps/place/Alpine+Roofing/data=!4m7!3m6
!1s0x89d4cc9b150c8a9f:0xb0887b40788685bd → cid (_CID_RE, :30)
!8m2!3d43.656728!4d-79.338035 → lat / lng (_COORD_RE, :28)
!16s%2Fg%2F1tk68nss → (unused feature id)
!19sChIJn4oMFZvM1IkRvYwGeEB7iLA → place_id (_PLACEID_RE, :29)
```

Coordinate fallback is `@lat,lng` (`_ATLL_RE`, `:31`) — present on some URL shapes. `maps_link` is the href with the query string stripped (`:47`). This is verified end-to-end by `tests/test_parse.py:21-33`.

**Sponsored** (`:144-147`) — `*[@aria-label="Sponsored"]` or an exact-text `span`. The fixture contains 2 ads out of 8 cards, asserted at `tests/test_parse.py:51`.

**Rating** (`:150-160`) — `span.MW4etd` digits, else the first `span[role="img"]` whose `aria-label` matches `([\\d.]+)\\s\\star`.

**Review count** (`_extract_review_count`, `:187-200`) — `span.UY7F9` or `span.RDAPeE` (the `(1,234)` element), else `([\\d,]+)\\s\\star` from a star aria-label; commas stripped. Best-effort: the feed often omits it entirely, which is why `review_count` is in `UPGRADEABLE_FIELDS`.

**Website** (`:170-173`) — `a[data-value="Website"]/@href`, **rejecting** hrefs starting with `/acлк`, `/url`, or `https://www.google`. — those are ad-click redirects, not the business's site. `tests/test_parse.py:63-65` asserts every organic card yields an `http...` website while the two ad cards yield none.

**Category + address** (`_extract_category_address`, `:209-233`) — the genuinely heuristic part. Google renders these as separator-joined text inside `div.W4Efsd` rows, so:

```
_info_rows(card) :203-206 innermost W4Efsd rows only (no nested W4Efsd)
for each row's text:
    skip if empty
    skip if fullmatch [\\d.]+ (a bare rating row)
    skip if it contains the already-found phone (the "Open · phone" row)
    skip if it matches a status word AND has no separator char
    split on . . . (_SEP_CHARS, :38)
    drop parts containing a status word (open/closed/closes/opens/24 hours/..., :34-37)
    category = parts[0]; address = parts[-1] if len(parts) > 1 else ""
    skip if the phone ended up inside address
    return (category, address)
```



**Phone** ( :166-167 , :180-183 ) — `span.UsdLK` is the stable element; the fallback `_phone_from_rows` ( :236-244 ) regex-scans info rows for `\+?\d{6,}\d` and accepts only matches with  $\geq 7$  digits.

## 8.2 The collection loop

`scrape_domain_progressive` main loop ( `scraper.py:629-690` ):

```
while worker._running and matched < target and (matched + len(pending)) < collect_cap:
    _wait_if_paused()                                :630    pause honoured every round
    cards = parse_feed_html(_feed_html(driver))        :634    full re-parse of the whole feed
    for card in cards:
        skip _sponsored                               :637-638
        skip if _key seen / missing                   :639-643
        seen.add(_key); new_this_round += 1
        if not cheap_pass(card, spec): continue       :645-646    ← reject before spending anything
        if _detail_wants(card, fields, spec):          :648-649
            pending.append((card, _href))             ← defer to pass 2
        else:
            _wait_if_paused(); bail if stopped         :654-656    ← pause during enrichment too
            if enrich_active: do_enrich(card)          :657-658
            if full_pass(card, spec): emit(card); matched+=1 :659-661
            break out early if target / collect_cap reached :662-663
    ...termination checks...                          :670-684
    _scroll_for_more(driver)                          :690
```

Re-parsing the *entire* feed each round (rather than only new cards) is intentional: it costs one lxml pass over a few hundred KB and makes the loop stateless with respect to Maps' DOM recycling. The `seen` set makes it idempotent.

The `_wait_if_paused()` at :654 — inside the per-card branch, before enrichment — is what makes Pause feel responsive: enrichment can take seconds per card, and without this the user would wait out the whole batch.

---

## FLOW 9 — The two-gate filter pipeline

`core/filters.py` splits filtering by **the cost of the data it needs**.

```
normalize_spec(spec)                                filters.py:40-54
  coerces every field: floats, ints, bools, and comma/newline-split lowercase term lists
  → always returns all 11 keys, so downstream code never guards for absence
```

| GATE  | FUNCTION                          | NEEDS                                                               | APPLIED AT                                                                  |
|-------|-----------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------|
| cheap | <code>cheap_pass</code> (:80-105) | card-level only: rating, website presence, name text, category text | <code>scraper.py:645</code> — during collection, before any spend           |
| full  | <code>full_pass</code> (:108-126) | fully populated: phone, email, review_count (+ re-runs cheap)       | <code>scraper.py:659</code> and <code>:720</code> — immediately before emit |

`full_pass` calls `cheap_pass` first (:112-113). That re-check matters: a detail visit or enrichment can *add* a website to a card that had none, and the re-check is what stops such a record from slipping past a `require_no_website` filter.

**Cost-declaration functions** — the scraper asks the spec what work it implies, so filters never trigger work they don't need:

```
needs_reviews(spec) # :67-69  min_reviews>0 or max_reviews>0 → detail visit for review_count
needs_phone(spec)   # :72-73  require_phone                → detail visit for phone
needs_email(spec)   # :76-77  require_email                → website fetch
is_active(spec)     # :57-64  any filter set at all         → used only for the "none matched" log
```

Semantics worth knowing, all asserted in `tests/test_enrich_filters.py:79-116`:

- Missing rating is coerced to `0.0` (`_to_float`, :18-23), so `min_rating > 0` **rejects unrated businesses** — including brand-new listings.
- `max_reviews = 0` means *no cap*, not *zero reviews* (:45), and the UI shows `"No cap"` as the spinbox special value (`screen_input.py:325`).
- Include lists are **any-match** (`any(t in name ...)`, :94); exclude lists are **any-match reject** (:96). Both are substring, case-insensitive.
- `_terms` (:32-38) splits on commas *and* newlines, so a filter term cannot itself contain a comma.

## FLOW 10 — Work planning: what forces a detail visit

`_detail_wants(card, fields, spec)` (`scraper.py:524–543`) is the cost-control centre of the whole engine. It returns the list of fields to fetch from the place page — **empty means no visit**.

```
want = [f for f in fields if f in DETAIL_ONLY_FIELDS]          # hours, review_1..3 → always a visit
if (phone requested or filter needs phone) and not card.phone: want += phone
enrich_needed = any enrich field requested or filter needs email
if (website requested or enrich_needed) and not card.website: want += website
if (review_count requested or filter needs reviews) and not card.review_count: want += review_count
if rating requested and not card.rating: want += rating
if want: # a visit is happening anyway...
    want += [address, category] (those that were requested) # ...so upgrade these for free
return dedup(want)
```

Three properties of this design:

1. **Conditional on the card.** If the feed already gave us the phone, no visit — this is why a typical service-business query does ~0 detail visits.
2. **Filter-aware.** `needs_phone` / `needs_email` / `needs_reviews` make a *filter* pull the data it needs even when the user didn't tick the column.
3. **Opportunistic upgrade.** `address` and `category` come from the heuristic feed parse; the detail page has them verbatim. They never *cause* a visit but are always refreshed by one — the "free-rider" rule at `:539–542`, applied via `UPGRADEABLE_FIELDS (:44)` at `:713–717`.

`_enrich_wants (:546–550)` is simpler and gated on having a website at all:

```
want = [f for f in fields if f in ENRICH_FIELDS]
if needs_email(spec) and "email" not in want: want.append("email")
return want if card.get("website") else [] # no website → nothing to fetch
```

`enrich_active (:601)` is the run-level version of the same question, used to skip `do_enrich` entirely when nothing needs it.

## FLOW 11 — Detail-page pass

Pass 2 runs after the collection loop ( `scraper.py:703-722` ) over the `pending` list, in the **same tab** — `driver.get(href)` navigates away from the feed, which is why this cannot be interleaved with collection.

```
for idx, (card, href) in enumerate(pending, 1):
    stop if not _running or matched >= target      :706-707    ← never over-fetch
    _wait_if_paused()                             :708
    log "Fetching details idx/total: <name>"       :709-710
    want = _detail_wants(card, fields, spec)       :711        ← recomputed, card may have changed
    detail = _extract_detail(driver, href, want)    :712
    merge: detail-only fields and UPGRADEABLE fields overwrite; others fill only if empty :713-717
    if enrich_active: do_enrich(card)             :718-719
    if full_pass(card, spec): emit(card); matched+=1 :720-722
```

`_extract_detail(driver, href, fields)` (:449-496):

```
driver.get(href)
WebDriverWait(8s) for h1.DUwDvf | h1.fontHeadlineLarge | h1[class*=fontHeadline] :454-456
sleep(T_DETAIL_LOAD = 0.5)                                                         :457
address      button[data-item-id='address'] div.Io6YTe | button[data-tooltip='Copy address'] .Io6YTe
phone        button[data-item-id='phone'] .Io6YTe | ... , fallback a[href^='tel:'] :465-474
website      a[data-item-id='authority'] | a[aria-label='website'] → @href          :475-478
category     _extract_category: button[jsaction='category'] (text < 80 chars)    :331-340
rating/count _extract_rating_and_count: parse "N stars" / "N reviews" aria-labels :428-446
hours        _extract_hours: click the hours button, read table.eK4R0e rows      :367-387
reviews      _extract_reviews: open the Reviews tab, read up to 3 blocks          :404-425
return only truthful values                                                         :496
```

Two sub-flows are click-driven and therefore the most fragile:

- **Hours (:367-387)** — find the button ( `data-item-id='oh'` , or `aria/tooltip` containing "Hours", :343-354 ), seed a value from its text or `aria-label` (with the `Hours:` prefix stripped by `_hours_from_label` , :357-364 ), then JS-click it and read `table.eK4R0e tr / tr.y0skZc` rows, joining cells with `" : "` and rows with `" ; "` → `"Monday: 9 AM-5 PM; Tuesday: ..."`. If the click or table read fails, the seeded single-line value survives.
- **Reviews (:390-425)** — `_open_reviews_tab` clicks `button[aria-label="Reviews"]` or `button[data-tab-index="1"]` (verifying "review" appears in the label), waits ≤4 s for `div.jftiE / div[data-review-id]` , then takes the first 3 blocks and formats each as `author | rating-aria-label | date | body` , dropping empty parts.

All element access goes through the `_first_text` / `_first_attr` / `_xpath_text` helpers (:290-323), which take a comma-separated selector string, try each in order, and return `""` rather than raising. Every click in the codebase is `driver.execute_script("arguments[0].click();")` (:125, :375, :396) — never `.click()` — because Maps overlays intercept native clicks.

## FLOW 12 — Website enrichment (email + socials)

Enrichment never uses the browser. `core/enrich.py` fetches the business site over plain HTTP, which is why it is fast enough to be worth doing at all and why its core is a pure function.

```
enrich_website(url, timeout=8.0, fields)                enrich.py:148-179
"" → all-empty dict                                   :152-154
add https:// if the URL has no scheme                  :155-156
_fetch(url) → html (never raises: except → "")         :159-162
extract_contacts(html, url)                            :164
if no email yet:                                       :167-177
    _guess_contact_url(html, url) → first href matching contact|about|reach|connect
    _fetch that page → extract → fill only the still-empty keys
return only the requested 'fields'                    :179
```

`_fetch` (:59-74): spoofed desktop-Chrome UA (:21-24), `Accept-Encoding: gzip` with manual `gzip` decompression (:68-72), `resp.read(1_500_000)` hard cap, `decode(errors="replace")`.

`extract_contacts(html, base_url)` (:95-145) — pure, no network:

### Email selection order

```
1. every mailto:... href (highest trust)                :108-111
2. every _EMAIL_RE match in the raw HTML                :112-115
3. _clean_email on each:                                :77-92
    lowercase, strip trailing dots, must fullmatch the email regex
    reject asset extensions (.png/.jpg/.css/.js/.woff...) :81-82
    reject junk domains: sentry, wixpress, godaddy, squarespace,
    shopify, cloudflare, googleapis, gstatic, schema.org, w3.org,
    jquery, bootstrapcdn, fontawesome, example., yourdomain,
    domain.com, email.com, test.com, mailerlite, wordpress.org :30-35, :84-85
    reject a local part that is 20+ hex chars (Sentry/analytics IDs) :87-88
    reject a local part with no letters                  :90-91
4. de-dupe preserving order, then STABLE-sort own-domain first :117-128
5. out["email"] = ordered[0]                             :129-130
```

Own-domain preference (`_same_domain`, :123-125) compares against `base_url`'s host with a `www.` prefix strip (:101-103) and accepts subdomains via `endswith("." + host)`. The stable sort means that among same-domain candidates, `mailto:` links still win. `tests/test_enrich_filters.py:61-69` pins the lookalike-domain case (`notwisdom.com` must not be preferred for `www.wisdom.com`).

**Socials** (:132-143) — one regex per platform (:39-45, including `x.com` for Twitter and

`?.linkedin.com/(company|in|pub)`), then per candidate:

```
strip trailing "'.,);
reject share/intent widgets: /sharer, /share.php, /intent/, /plugins/, /dialog/, /tr?,
                             facebook.com/(tr|plugins|sharer), /hashtag/         :50-54
reject bare roots: empty path, or path == "home" / "share"                     :138-141
first survivor wins per platform
```

The comment at :48-49 documents a deliberate non-rule: **do not reject on a bare** `=`, because legitimate profiles use query strings (`facebook.com/profile.php?id=...`, `youtube.com/channel/x?sub_confirmation=1`). That case is pinned by `tests/test_enrich_filters.py:49-58`.

Call site: `do_enrich` (`scraper.py:612-619`) — computes `_enrich_wants`, calls `enrich_website`, and copies only truthful values onto the card. It runs in **two** places (:658 for card-complete records, :718 after a detail visit) and is **fully serial**: worst

case  $\approx 2 \text{ fetches} \times 8 \text{ s} = 16 \text{ s}$  per record. That is the dominant cost of any email-enabled run and the first item on `TODO.md:22`.

---

## FLOW 13 — Emit: worker thread → GUI thread → table

```
emit(card)                                     scraper.py:603-610
    emitted += 1
    record = _card_to_record(card, fields, domain, area)      :500-509
        {f: card.get(f, "") for f in fields} + _domain, _area, _href
        + explicit domain/area values if those pseudo-columns were requested
    result_signal.emit(record)                        → ResultsScreen.add_table_row
    progress_signal.emit(_progress_base + emitted)     → ResultsScreen.update_progress
    log_signal.emit(f"#{n} {name[:50]}", "done")       → ResultsScreen._on_log
```

`_card_to_record` (:500-509) **projects** the card onto exactly the requested fields — the emitted dict is a new object, so later mutation of `card` cannot race with the GUI thread. `_domain`, `_area`, `_href` are always attached regardless of the user's column choice, because the UI needs them for row actions and export labelling.

Receiving side, `add_table_row` (`screen_results.py:354-383`):

```
self.results.append(data)                      ← the in-memory export buffer
insertRow
for each column field:
    domain/area columns read data[field] then fall back to data[_domain]/[_area] :360-365
    empty → "-"                               :366
    display = text[:77] + "..." if len > 80  :367
    SortableItem(display)
        setData(Qt.UserRole + 1, full_text) ← full value kept for search + tooltip :372
        col 0 only: setData(Qt.UserRole, data) ← the whole record, for row actions :375-376
if a search filter is active and the row doesn't match → hide it; else scrollToBottom :379-382
_update_row_count()
```

`SortableItem` (:16-31) overrides `__lt__` to compare numerically when both cells parse as numbers (commas stripped) and case-insensitively otherwise — so Rating and Review Count sort as numbers, not as text. Sorting is enabled **only when idle** (:218, :228): re-sorting while rows stream in would fight `scrollToBottom` and reorder under the user's cursor.

`update_progress` (:348-351) drives the 2-px progress bar, clamping the value to `_max_results` and re-asserting the maximum each time (cheap, and keeps the bar correct after a per-task reset).

`_on_log` (:339-346) is deliberately lossy: "active" messages replace the status line, "done" messages are shown only if they start with `#` (the per-result lines), truncated to 48 characters. There is no scrolling log pane; the status line is a single-slot display.

## FLOW 14 — The five ways a task terminates

`scrape_domain_progressive` can leave its loop five ways ( `scraper.py:629–690` ), and the distinction matters because it decides both the log wording and whether the run continues.

| # | CONDITION               | CODE                                                                                                                                        | COMPLETED                                                     |
|---|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| 1 | target reached          | <code>matched &gt;= target</code> ( :629, :662, :667 )                                                                                      | <code>True</code> if <code>max_results &gt; 0</code> ( :730 ) |
| 2 | collection cap reached  | <code>matched + len(pending) &gt;= collect_cap</code>                                                                                       | as above                                                      |
| 3 | Google says end-of-list | <code>_is_end_of_list()</code> and no new cards this round ( :679–681 ) → <code>ended = True</code>                                         | <code>True</code>                                             |
| 4 | stall / no growth       | <code>stall &gt;= max_stall (6)</code> <b>or</b> <code>time.time() - last_growth &gt; 120 s</code> ( :682–684 ) → <code>ended = True</code> | <code>True</code>                                             |
| 5 | user stopped            | <code>worker._running</code> went False ( :629, :631–632, :665–666, :700–701 )                                                              | <code>False</code>                                            |

```
completed = ended or (max_results > 0 and matched >= max_results)    # :730
return matched, completed
```

Also note the feed-recovery branch at :686–689: if `_get_feed()` returns `None` mid-run (Maps navigated away, or the container was replaced), the task re-GETs the search URL, sleeps `T_SEARCH_LOAD`, and `continue`s — the `seen` set means already-collected listings are not re-emitted.

Two "nothing found" reports are emitted from inside this function:

- `not seen` at all → `_debug_dump("noresults_...")` + a log line + `return 0, True` ( :692–698 ).
- `matched == 0` while filters are active → an explicit *"scanned N but none matched your filters"* line ( :724–728 ), so a zero-result run distinguishes "Google gave us nothing" from "your filters rejected everything".

`stall` counts rounds where the parse produced no unseen cards; each such round also re-emits a *"Scanned N, matched M so far..."* status ( :674–677 ) so a long stall still looks alive.



## FLOW 15 — Multi-task handshake

In multi mode the worker does **not** roll straight into the next `(domain, area)`. It hands control to the GUI and blocks until acknowledged, so the UI can export and clear the table without racing incoming rows.

| WORKER (scraper.py:860-864)                                                                            |   | GUI (screen_results.py:528-538)                                                 |
|--------------------------------------------------------------------------------------------------------|---|---------------------------------------------------------------------------------|
| <code>_continue_event.clear()</code>                                                                   |   |                                                                                 |
| <code>domain_finished_signal.emit(<br/>    domain, area, count,<br/>    max_results, hit_limit)</code> | → | <code>_on_domain_finished(...)</code>                                           |
|                                                                                                        |   | <code>_notify_task_export(rows=list(self.results))</code>                       |
|                                                                                                        |   | → build CSV for this task                                                       |
|                                                                                                        |   | → toast with the path                                                           |
|                                                                                                        |   | <code>self.results = []</code> ← buffer reset                                   |
|                                                                                                        |   | <code>table.setRowCount(0)</code> ← table cleared                               |
|                                                                                                        |   | counters/status reset                                                           |
|                                                                                                        | ← | <code>worker.continue_next_domain()</code> → <code>_continue_event.set()</code> |
| <code>next task</code>                                                                                 |   |                                                                                 |

`hit_limit (:861)` is `max_results > 0 and count >= max_results` — it exists purely so the toast can say "50 of 50 collected" versus "37 of 50 requested — Google Maps had fewer" (`_export_summary`, `screen_results.py:476-486`).

The wait loop is `while _running and not _continue_event.wait(timeout=0.2)` — polling with a timeout rather than blocking forever, so `stop()` (which sets both `_running = False` and `_continue_event.set()`, `:776-779`) always releases it.

## FLOW 16 — CSV export

Export is **automatic per task**, plus manual on demand. Both go through the same function.

```
_notify_task_export(domain, area, count, hit_limit, rows=None)    screen_results.py:500-511
message = _export_summary(...)    ← wording depends on count vs max vs hit_limit
rows    = self.results if rows is None
if count > 0 and rows:
    export_csv(_export_rows(rows), domain, area, self.fields, self._export_dir)
    on exception → append "(CSV save failed: ...)" to the toast    :509-510
_show_toast(message, filepath)    ← 6 s single-shot QTimer, :513-522
```

`_export_rows` ( :488-498 ) copies each record and back-fills the `domain` / `area` pseudo-columns from `_domain` / `_area`, so those columns are never blank in the file.

`export_csv` ( `core/exporter.py`:31-59 ):

```
output_path ends with .csv    → use it verbatim
else                          → <output_path or ". ">/<domain>_in_<area>.csv
                               lowercased, spaces → underscores    :42-45
headers = [FIELD_LABELS[f] for f in fields if f in FIELD_LABELS]    :47
open(filepath, "w", newline="", encoding="utf-8-sig")                :49
csv.DictWriter(fieldnames=headers, extrasaction="ignore")            :50
writeheader(); one row per record, mapping field key → label        :52-57
return filepath
```

Three deliberate choices: `utf-8-sig` (Excel reads UTF-8 correctly only with a BOM), `newline=""` (the csv module's requirement — otherwise Windows writes `\r\r\n`), and `extrasaction="ignore"` (bookkeeping keys like `_href` can never leak into the file).

Call sites:

| TRIGGER               | PATH                                                                | ROWS EXPORTED                                                           |
|-----------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------|
| task finished (multi) | <code>_on_domain_finished</code> ( :528-538 )                       | that task's rows, then the buffer is cleared                            |
| run finished (single) | <code>on_done</code> ( :540-562 )                                   | everything collected                                                    |
| user stopped          | <code>on_done</code> via <code>_stopped_by_user</code> ( :542-548 ) | whatever was collected — <i>"Stopped — partial results saved"</i>       |
| Export CSV button     | <code>_on_export_clicked</code> ( :568-573 )                        | the current buffer, using <code>_current_task()</code> for the filename |

`_current_task()` ( :468-474 ) derives the label from the **last collected record's** `_domain` / `_area`, falling back to the first requested pair — so a mid-run manual export is named after the task actually in progress.

## FLOW 17 — Pause, Stop, Abort, Close (the shutdown ladder)

Selenium calls block the worker thread. A co-operative flag cannot interrupt an in-flight page load, so the codebase implements a deliberate escalation.

### Pause / Resume

```
pause()    scraper.py:797-800    _paused = True; _pause_lock.clear(); paused_signal.emit(True)
resume()   :802-805             _paused = False; _pause_lock.set();  paused_signal.emit(False)
_wait_if_paused() :807-809     while _running and not _pause_lock.is_set(): sleep(0.15)
```

`threading.Event` used inverted (`set` = running) and initialised set ( :766-767 ). Pause is checked at four points — top of the collection round ( :630 ), before per-card enrichment ( :654 ), in the detail pass ( :708 ) — which bounds pause latency to roughly one card's work, not one round's.

### Stop → Abort → Terminate

```
stop()      scraper.py:776-779    _running = False; _pause_lock.set(); _continue_event.set()
                                         ← releases every wait so the loop can observe the flag
abort()     :781-795             stop(), then driver.quit() FROM THE CALLING (GUI) THREAD
                                         ← makes the in-flight Selenium call fail fast so run() unwinds
```

The docstring at :782-789 states the reasoning: without the external `driver.quit()`, closing the window during a scrape leaves the QThread alive and an orphaned `chrome.exe` behind — the app appears to hang on exit and Chrome processes pile up.

`MainWindow.shutdown_worker()` ( `app.py:489-508` ) is the ladder:

```
worker missing or not running    → return
_stopped_by_user = True          ← so on_done exports partial results
worker.stop(); wait(5000)         → return if it exits          (clean)
worker.abort(); wait(5000)        → return if it exits          (browser force-closed)
worker.terminate(); wait(2000)    (last resort)
```

It is reachable from three places, and is idempotent because of the first guard: `closeEvent` ( :510-512 ), `aboutToQuit` ( :546 ), and `SIGINT` ( :538-542 ).

`ResultsScreen.stop_worker()` ( :314-320 ) is the in-app Stop button path: sets `_stopped_by_user`, calls `worker.stop()`, disables both buttons, shows "Stopping..." — it does **not** abort, so a Stop during a page load waits for that load to finish or time out ( $\leq 30$  s per `set_page_load_timeout`).

Regardless of path, `run()`'s `finally` ( :869-876 ) quits the driver, clears `_driver`, and emits `done_signal` — so the UI always returns to idle mode.

## FLOW 18 — Failure paths and the debug dump

| FAILURE                       | DETECTION                                                                                    | RESPONSE                                                                                     |
|-------------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Chrome/driver won't start     | exception in <code>get_driver</code>                                                         | <code>error_signal</code> → <code>on_error</code> ( <code>screen_results.py:564–566</code> ) |
| Consent page                  | URL contains <code>consent.google.com</code>                                                 | up to 2 dismissal attempts, then reported ( <code>:563–576</code> )                          |
| CAPTCHA / "unusual traffic"   | URL <code>sorry/index</code> or body text                                                    | <code>_debug_dump("blocked_...")</code> , task returns <code>(0, True)</code>                |
| No feed in 12 s               | <code>WebDriverWait</code> timeout                                                           | <code>_debug_dump("nofeed_...")</code> + advice to inspect the dump                          |
| Feed present, zero cards      | <code>not seen</code> after the loop                                                         | <code>_debug_dump("noresults_...")</code>                                                    |
| Feed vanished mid-run         | <code>_get_feed()</code> is <code>None</code> ( <code>:686</code> )                          | re-GET the search URL and continue                                                           |
| Detail page failure           | <code>except</code> in <code>_extract_detail</code> ( <code>:494–495</code> )                | <code>out["_error"]</code> ; the card keeps its card-level values                            |
| Website unreachable / hostile | <code>except</code> in <code>_fetch</code> ( <code>:161–162</code> , <code>:175–177</code> ) | enrichment yields <code>""</code>                                                            |
| CSV write failure             | <code>except</code> in <code>_notify_task_export</code> ( <code>:509–510</code> )            | appended to the toast text                                                                   |
| Slot raises in the GUI        | <code>sys.excepthook</code> ( <code>app.py:515–522</code> )                                  | traceback printed, app stays alive                                                           |

`_debug_dump(driver, tag)` (`scraper.py:512–520`) writes `debug/<sanitised-tag>.png` and `.html` — tag sanitised to `[A-Za-z0-9_]` and truncated to 40 chars. The `debug/` directory is git-ignored (`.gitignore:15`) **except** the committed fixture `debug/initial_Roofing_company.html`, which `tests/test_parse.py:14–18` depends on.

## FLOW 19 — Build & packaging pipeline

`BUILD_EXE.bat` `<version>` `[debug]` (196 lines) is the real build entry point:

```
[version] arg 1 or prompt; strips a leading v; must match ^\d+\.\d+\.\d+$ :14-32
          OUTNAME = MapHarvest-<version>          (+ "-debug" in debug mode)
          debug mode → --console instead of --windowed, so startup errors are visible :38-43
[python] prefer venv\Scripts\python.exe, else system "python" :51-63
[1/5]    ensure PyInstaller; ensure PyQt5/selenium/undetected_chromedriver/lxml :67-88
[2/5]    generate app_icon.ico via tools/make_icon.py if missing :91-103
[3/5]    tools/gen_version_file.py <version> build\version_info.txt :106-113
[4/5]    PyInstaller --onefile --windowed --name <OUTNAME>
          --icon app_icon.ico --version-file build\version_info.txt
          --distpath releases --workpath build\work --specpath build
          --collect-all undetected_chromedriver --collect-all selenium
          --hidden-import lxml.etree --hidden-import lxml._elementpath
          --exclude-module tkinter/matplotlib/numpy :121-154
[5/5]    report path/version/size, offer to open releases\ :159-179
:buildfailed actionable tips (close the running EXE, AV interference, debug build) :181-195
```

Why each PyInstaller flag is there:

- `--collect-all undetected_chromedriver` — uc ships data files and patches a driver binary at runtime; a plain import-graph scan misses them.
- `--hidden-import lxml.etree`, `lxml._elementpath` — lxml's C extensions are invisible to static analysis, and `core/parse.py` is useless without them.
- `--exclude-module tkinter/matplotlib/numpy` — none are used; excluding them cuts tens of MB.
- `--onefile` — one artifact to hand over; the trade-off is a temp-dir extraction on each launch.

`tools/gen_version_file.py` (75 lines) writes a `VSVersionInfo` resource so the EXE has real Windows file properties; it re-validates the version ( :55-58 ) and creates the output directory ( :61 ).

`tools/make_icon.py` (189 lines) is a dependency-free rasteriser: signed-area point-in-triangle and point-in-circle tests ( :28-67 ) sampled at **4x supersampling** ( :70-99 ) to antialias, then a hand-written ICO container (BITMAPINFOHEADER + bottom-up BGRA + AND mask, :103-147 ) at seven sizes, plus a minimal hand-rolled PNG writer using `zlib` + CRC32 chunks ( :150-166 ). No Pillow.

**Runtime requirement of the built EXE:** Google Chrome must be installed on the target machine — undetected-chromedriver downloads/patches the *driver*, not the browser.

## Performance model: where the seconds go

| WORK UNIT                     | COST                                 | WHERE                         |
|-------------------------------|--------------------------------------|-------------------------------|
| Search navigation + consent   | ~1–4 s once per task                 | :559–570                      |
| Feed parse (whole feed, lxml) | milliseconds                         | parse_feed_html               |
| One scroll round              | 0.4–5 s until growth is detected     | _scroll_for_more ( :271–286 ) |
| One detail visit              | ~1–3 s (8 s wait cap + 0.5 s settle) | _extract_detail               |
| Hours sub-flow                | +~0.25 s and one click               | :375–376                      |
| Reviews sub-flow              | +~0.35 s, one click, ≤4 s wait       | :396–412                      |

| WORK UNIT              | COST                             | WHERE                        |
|------------------------|----------------------------------|------------------------------|
| One website enrichment | 0–16 s (2 fetches × 8 s timeout) | <code>enrich.py:59–74</code> |

Practical consequences:

- **Fast path** — card-only fields, no filters requiring late data: zero detail visits. `TODO.md:10` records a measured service query at ~2 s for 4 results.
- **POI queries** are slower not by field count but because POI cards routinely omit phone and website, so `_detail_wants` fires per card.
- **Enrichment dominates everything else** when enabled, and it is serial.
- Timing constants live in one place (`scraper.py:36–39`); `T_AFTER_SCROLL` is currently unused (growth detection replaced the fixed sleep).
- Google caps a single search at roughly 120 results regardless of scrolling — hence the "split big cities into sub-areas" idea at `TODO.md:24`.

## Selector registry (the fragile surface)

Every Google-DOM dependency in one table. When a scrape suddenly returns empty fields, this is the list to re-verify against a fresh `debug/` dump.

### Feed (lxml, `core/parse.py`)

| TARGET         | SELECTOR                                           | LINE                  | FALLBACK                                                                    |
|----------------|----------------------------------------------------|-----------------------|-----------------------------------------------------------------------------|
| card container | <code>div.Nv2PK</code>                             | <code>:252</code>     | <code>div[role=article]</code> holding a place anchor ( <code>:255</code> ) |
| anchor / name  | <code>a.hfpxzc</code> → <code>@aria-label</code>   | <code>:124–132</code> | <code>a[href*="/maps/place"]</code> ; <code>div.qBF1Pd</code>               |
| sponsored      | <code>[aria-label="Sponsored"]</code>              | <code>:144–146</code> | exact-text <code>span</code>                                                |
| rating         | <code>span.MW4etd</code>                           | <code>:150</code>     | <code>span[role=img]@aria-label</code> "N stars"                            |
| review count   | <code>span.UY7F9</code> , <code>span.RDApEe</code> | <code>:189</code>     | aria-label "N reviews"                                                      |
| phone          | <code>span.UsdLK</code>                            | <code>:166</code>     | regex over <code>div.W4Efsd</code> rows                                     |
| website        | <code>a[data-value="Website"]@href</code>          | <code>:170</code>     | none (ad hrefs rejected)                                                    |
| info rows      | innermost <code>div.W4Efsd</code>                  | <code>:203–206</code> | none                                                                        |

### Detail page (Selenium, `core/scraper.py`)

| TARGET              | SELECTOR                                                                                                                                                                    | LINE                  |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| page-ready sentinel | <code>h1.DUwDvf</code> , <code>h1.fontHeadlineLarge</code> , <code>h1[class*=fontHeadline]</code>                                                                           | <code>:454–456</code> |
| address             | <code>button[data-item-id='address']</code> <code>div.Io6YTe</code> , <code>button[data-tooltip='Copy address']</code> <code>.Io6YTe</code>                                 | <code>:459–464</code> |
| phone               | <code>button[data-item-id^='phone']</code> <code>.Io6YTe</code> , <code>button[data-tooltip='Copy phone number']</code> <code>.Io6YTe</code> , <code>a[href^='tel:']</code> | <code>:465–474</code> |

| TARGET         | SELECTOR                                                                                                                                                                                 | LINE                  |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| website        | <code>a[data-item-id='authority'], a[aria-label*='website']</code>                                                                                                                       | :475-478              |
| category       | <code>button[jsaction*='category'], button.DkEaL[jsaction*='category']</code>                                                                                                            | :332                  |
| rating/reviews | <code>div.F7nice, div.fontBodyMedium, button / span[aria-label*='review']</code>                                                                                                         | :430-445              |
| hours button   | <code>button[data-item-id='oh'], button[aria-label*='Hours'], button[data-tooltip*='hours' 'Hours']</code>                                                                               | :344-349              |
| hours table    | <code>table.eK4R0e tr, tr.y0skZc</code>                                                                                                                                                  | :378                  |
| reviews tab    | <code>button[aria-label*="Reviews"], button[data-tab-index="1"]</code>                                                                                                                   | :391                  |
| review block   | <code>div.jftiE, div[data-review-id]</code>                                                                                                                                              | :410-414              |
| review parts   | author <code>div.d4r55 / span.dwiWPf</code> ; stars <code>span.kvMYJc / span[role=img]</code> ; date <code>span.rsqaWe / span.dehysf</code> ; body <code>span.wiI7pd / div.MyEneD</code> | :416-419              |
| feed container | <code>div[role=feed], div[aria-label*="Results for" "Search results"]</code>                                                                                                             | :153-157,<br>:228-232 |
| loading        | <code>div[role=progressbar], div.section-loading, div.loading</code>                                                                                                                     | :180                  |
| end of list    | text "You've reached the end" / "reached the end" / "end of the list"                                                                                                                    | :203-208              |
| consent        | 5 XPath on Accept all / Reject all / I agree                                                                                                                                             | :114-120              |

Two classes of dependency here: **obfuscated class names** (`Nv2PK`, `hfpxzc`, `MW4etd`, `UsdLK`, `Io6YTe`, `eK4R0e`, `jftiE`...) which Google rotates without notice, and **semantic attributes** (`role="feed"`, `data-item-id`, `aria-label`, `data-value`) which are far more stable. The codebase consistently prefers semantic selectors as the primary and class names as accelerators — except in the feed parser, where obfuscated classes are primary because there is no semantic equivalent.

## Test inventory

Both suites are plain `python -m` scripts with `assert`s — no pytest dependency, no network, no browser.

```
venv/Scripts/python.exe -m tests.test_parse
venv/Scripts/python.exe -m tests.test_enrich_filters
```

**tests/test\_parse.py** — runs against the real feed capture `debug/initial_Roofing_company.html`. **Caveat:** that file is *untracked* (`.gitignore:13` ignores `debug/`), so on any clone the feed test silently `return`s at `:37-39` and the script still prints "ALL TESTS PASSED" — see [A11](#). The assertions below therefore only hold where the capture exists locally:

| ASSERTION                                                               | LINE   |
|-------------------------------------------------------------------------|--------|
| URL decoding: <code>lat/lng/place_id/cid + place_key</code> == Place ID | :21-33 |
| exactly 8 cards parsed                                                  | :44    |
| 8 unique <code>_key</code> s (no dedup collision)                       | :46-47 |
| exactly 2 sponsored / 6 organic                                         | :49-52 |
| every card has name, rating, phone, category, address, coords, place_id | :54-61 |

| ASSERTION                                                                                 | LINE    |
|-------------------------------------------------------------------------------------------|---------|
| every <b>organic</b> card's website starts with <code>http</code> (ad redirects rejected) | : 63–65 |
| prints a formatted table for eyeball verification                                         | : 68–74 |

`tests/test_enrich_filters.py` — five cases:

| CASE                                                   | PINS                                                                                                                                                            |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_enrich_extract</code> (: 29–40)             | own-domain email preferred over a body-scraped one; share/intent socials skipped; wixpress/sentry/hex/asset junk rejected                                       |
| <code>test_enrich_empty</code> (: 43–46)               | empty HTML → all-empty dict, no raise                                                                                                                           |
| <code>test_social_with_query_string</code> (: 49–58)   | <code>profile.php?id=...</code> and <code>channel/...?sub_confirmation=1</code> survive the junk filter                                                         |
| <code>test_www_prefix_and_domain_pref</code> (: 61–69) | <code>www.</code> prefix strip; lookalike domain ( <code>notwisdom.com</code> ) not preferred                                                                   |
| <code>test_filters</code> (: 79–116)                   | <code>min_rating</code> (incl. empty→0), website presence both ways, name/category include+exclude, phone/email/min+max reviews, all <code>needs_*</code> flags |

**Coverage gap, by construction:** `core/scrapper.py` (877 lines — the orchestration, termination, budget and threading logic), `core/exporter.py`, `core/settings.py` and the entire `ui/` layer have no automated tests. Everything reachable without a browser is tested; everything requiring one is verified by the live smoke runs recorded in `TODO.md:10` and `TODO.md:19`.

## System audit — findings

**Method.** Five specialist passes (engine, extraction, data layer, UI, build+docs) produced candidate defects. Every candidate was then handed to an independent adversarial reviewer instructed to *refute* it by re-reading the code, defaulting to "not a defect" when uncertain. 15 of 25 candidates survived. Findings marked **reproduced** were additionally executed against the real code on this machine — the exact commands and output are quoted. Refuted candidates are listed at the end so the same ground isn't re-covered.

Nothing in the codebase was modified for this audit.

### Summary

| #  | SEVERITY | FINDING                                                                                                                                             | WHERE                                                         |
|----|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| A1 | HIGH     | A review-count span in the rating row makes <b>the rating become the category</b> and drops the address entirely ( <b>reproduced</b> )              | <code>core/parse.py:209–233</code>                            |
| A2 | HIGH     | <code>_extract_detail</code> swallows session-fatal errors, so a dead browser or mid-run CAPTCHA silently blanks or drops every remaining candidate | <code>core/scrapper.py:494–496</code>                         |
| A3 | HIGH     | Multi-task run <b>destroys</b> a task's rows when the CSV write fails                                                                               | <code>ui/screen_results.py:528–538</code>                     |
| A4 | HIGH     | A <code>:</code> in the search term writes the CSV into an NTFS alternate data stream — 0-byte visible file, success reported ( <b>reproduced</b> ) | <code>core/exporter.py:42–49</code>                           |
| A5 | MEDIUM   | The over-collect cap ends collection for good, then the UI blames Google (" <i>Google Maps had fewer</i> ")                                         | <code>core/scrapper.py:593–595</code> , <code>:667–668</code> |



| #   | SEVERITY | FINDING                                                                                                                                                                        | WHERE                                                                  |
|-----|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| A6  | MEDIUM   | <code>done_signal</code> always follows <code>error_signal</code> , so a real failure is overwritten by "Done — no results"                                                    | <code>core/scrapper.py:869-876</code>                                  |
| A7  | MEDIUM   | Worker exception in a multi-task run exports nothing and reports "Done — all searches scraped"                                                                                 | <code>ui/screen_results.py:556-557</code>                              |
| A8  | MEDIUM   | Closing the window during enrichment escalates to <code>terminate()</code> , so the partial-results CSV is never written                                                       | <code>ui/app.py:489-508</code>                                         |
| A9  | MEDIUM   | One malformed link on one business website raises out of enrichment and <b>kills the entire run</b> ; non-HTTP schemes are fetched ( <b>reproduced</b> )                       | <code>core/enrich.py:164</code> , <code>:168</code>                    |
| A10 | MEDIUM   | <code>_guess_contact_url</code> takes the first match, so "About" — or a <code>connect.facebook.net</code> prefetch in <code>&lt;head&gt;</code> — beats the real contact page | <code>core/enrich.py:182-186</code>                                    |
| A11 | MEDIUM   | The parser test's fixture is untracked, so on every clone the test skips and still prints "ALL TESTS PASSED" ( <b>reproduced</b> )                                             | <code>tests/test_parse.py:37-39</code> ,<br><code>.gitignore:13</code> |
| A12 | LOW      | Phone fallback truncates the leading <code>()</code> and can surface the phone number as the category ( <b>reproduced</b> )                                                    | <code>core/parse.py:236-244</code>                                     |
| A13 | LOW      | Asset blocklist misses <code>.avif/.ttf/.pdf/.heic</code> , so an image filename is exported as the business email ( <b>reproduced</b> )                                       | <code>core/enrich.py:37</code>                                         |
| A14 | LOW      | Repeated identical searches silently overwrite the previous CSV ( <b>reproduced</b> )                                                                                          | <code>core/exporter.py:42-45</code>                                    |
| A15 | LOW      | Clicking a Recent Search silently wipes the multi-area list; multi-area searches can never round-trip                                                                          | <code>ui/screen_input.py:555</code>                                    |
| A16 | LOW      | The frozen EXE writes <code>debug/</code> relative to the working directory, so the folder the UI tells users to inspect may not exist                                         | <code>core/scrapper.py:512-520</code>                                  |
| A17 | LOW      | Two divergent build paths: <code>main.spec</code> is dead and stale, yet it is the only packaging artifact the README documents                                                | <code>main.spec</code> , <code>README.md:448</code>                    |
| A18 | LOW      | Every shipped EXE stamps "MIT Licensed" and the README carries an MIT badge, but the repo has no LICENSE file ( <b>reproduced</b> )                                            | <code>tools/gen_version_file.py:14</code>                              |
| A19 | LOW      | Dead code and constants duplicated across layers                                                                                                                               | several                                                                |
| A20 | LOW      | Documentation drift                                                                                                                                                            | <code>README.md</code> , <code>core/enrich.py:5</code>                 |

## A1 — A review count in the rating row destroys category and address · HIGH · wrong data · reproduced

`_extract_category_address` (`core/parse.py:209-233`) walks the leaf `div.W4Efsd` rows and **commits to the first row that yields any part** (the `return` at `:232`). Its rating-row guard at `:214` is `re.fullmatch(r"[\d.]+", text)` — it only recognises a bare numeric string.

Google renders the review count as a sibling span of `span.MW4etd` **inside the same leaf row** — that is precisely where `_extract_review_count` looks for it (`span.UY7F9` / `span.RDAPeE`, `:187-194`). When that variant is served, the rating row's text is `4.5(120)`. The parentheses defeat the `fullmatch` guard, `_STATUS_RE` doesn't match either, `_split_sep` returns `["4.5(120)"]`, and the function returns — so the genuine `Roofing contractor · 31 Carlaw Avenue` row is **never examined**.

**Reproduced** by adding to the project's own committed fixture only the span the module itself hunts for:

```
html = open('debug/initial_Roofing_company.html', encoding='utf-8', errors='replace').read()
mut = html.replace('<span class="MW4etd" aria-hidden="true">4.5</span>',
                  '<span class="MW4etd" aria-hidden="true">4.5</span><span class="UY7F9">(120)</span>', 1)
```

```
ORIGINAL card 1: rating='4.5' review_count='' category='Roofing contractor' address='31 Carlaw Avenue'
WITH THE SPAN: rating='4.5' review_count='120' category='4.5(120)' address=''

_info_rows() texts: ['4.5(120)', 'Roofing contractor · 31 Carlaw Avenue', 'Open 24 hours · (416) 469-1939']
                   ^ committed to this one           ^ the correct row, never reached
```

Three things make this worse than a cosmetic glitch:

- **Nothing recovers it.** `address` / `category` are in `UPGRADEABLE_FIELDS` (`core/scrapper.py:44`), but they are only appended to `want` when `want` is *already* non-empty (`:538-542`). In this variant `review_count` is card-available, so with a card-satisfiable field set no detail visit happens at all.
- **Category filters silently zero out the run.** `cheap_pass` (`core/filters.py:100-103`) runs before the detail pass, so any `category_include` term rejects every record.
- **The test suite cannot catch it.** `UY7F9` and `RDAPeE` occur **0 times** in the committed fixture (verified: `MW4etd`=8, `UY7F9`=0, `RDAPeE`=0), so all 8 captured cards are the count-less variant, and `tests/test_parse.py:54-61` never asserts `review_count` at all.

**Fix direction:** stop committing to the first row that yields parts. Skip rows with no alphabetic content (`if not re.search(r"[A-Za-z]", text): continue`) instead of relying on a bare-numeric `fullmatch`, and prefer a row that actually contains a separator.

## A2 — `_extract_detail` swallows session-fatal failures · HIGH · silent data loss

`core/scrapper.py:494-496` catches **every** `Exception` and stringifies it into `out["_error"]` — a key **no caller ever reads** (`_error` is written at `:495` and read nowhere; the merge at `:713-717` only copies keys present in `want`). Every Selenium exception derives from `Exception`, so `InvalidSessionIdException` / `WebDriverException` after Chrome dies — and equally a `TimeoutException` from the 8 s wait at `:454-456` when a CAPTCHA replaces the place page — each yields `{}` and the loop continues silently through all of `pending`.

The detail pass never calls `_page_blocked_reason`, unlike the initial load at `:572-576`.

Two outcomes, both silent:

- With a detail-dependent filter (`require_phone`, `min_reviews`), `full_pass` at `:720` rejects every remaining candidate — **permanently lost**, because collection is over and never resumes.
- Without one, rows emit with blank hours/reviews/phone, indistinguishable from absent data.

Either way the terminal state is *"Done — N businesses"* (`ui/screen_results.py:555`). A log-only mitigation would be invisible: `_on_log` (`:342`) drops `"done"` messages that don't start with `#`.

**Repro:** set `min_reviews > 0` — the feed carries no `review_count`, so `:534` queues every card. At *"Fetching details 5/100"*, kill `chrome.exe`: the pass finishes in seconds, no `error_signal` fires, and the run reports normal completion.

**Fix direction:** let session-fatal exceptions propagate (or return a typed sentinel); in the detail loop break on the first fatal result or on a non-empty `_page_blocked_reason(driver)`, emit `error_signal`, and return a "did not complete" state.

## A3 — Multi-task run destroys a task's rows when the CSV write fails · HIGH · data loss

`_on_domain_finished` (`ui/screen_results.py:528-538`) clears `self.results` (`:532`) and the table (`:533`) **unconditionally**. The export it just performed cannot report failure: `_notify_task_export` (`:500-511`) catches every `export_csv` exception, appends `"(CSV save failed: ...)"` to a toast that auto-hides after `TOAST_MS = 6000` (`:38`), and

returns nothing the caller inspects. The rows handed in at :531 are a throwaway copy — so on a failed write that (domain, area)'s records exist nowhere, and the run rolls on.

Realistic raise paths in export\_csv, whose filename stem is only strip().lower().replace(" ", "\_") (core/exporter.py:42-44):

- a domain like 24/7 locksmith, or an area like Kitchener/Waterloo → FileNotFoundError
- ? \* " < > | in either field → OSError 22
- the target CSV already open in Excel → PermissionError
- the export drive disconnected mid-run → OSError

**Verified by probe** (the two path-separator classes):

```
'bar / grill'      -> FileNotFoundError: ...\\bar_/_grill_in_toronto.csv
'hvac | plumbing' -> OSError 22:      ...\\hvac_|_plumbing_in_toronto.csv
```

The single-task path (on\_done, :549-555) does **not** clear self.results, so the rows survive there and Export CSV can still save them. Only the multi-task path is unrecoverable.

**Fix direction:** make \_notify\_task\_export return the written path (or a success flag) and clear the buffer only when a file was actually written; on failure keep the rows and show a persistent banner, not a 6-second toast. Independently, sanitise the filename stem (strip <>:"\\|?\* and control chars, cap length) and os.makedirs(output\_dir, exist\_ok=True) first.

#### A4 — A : in the search term hides the CSV in an NTFS stream · HIGH · silent data loss · reproduced

Worse than A3's exception cases, because **no exception is raised**. On NTFS, open("name:stream.csv", "w") writes to an *alternate data stream* of a file called name. The visible file is 0 bytes and the data is invisible to Explorer, Excel and every ordinary tool. export\_csv returns the path normally, so the toast reports "CSV saved to ...".

**Reproduced** (core/exporter.py:42-49):

```
export_csv([{{...}}, "who's #1: cafe", "Toronto", ["name"], tmpdir)
  returns <tmpdir>\\who's_#1:_cafe_in_toronto.csv      (reported as success)
  on disk <tmpdir>\\who's_#1                          0 bytes
```

A colon in a search phrase is not exotic — "open 24:7", "cafe: downtown", or any pasted title with a colon triggers it. In multi-task mode A3 then clears the rows.

**Fix direction:** same sanitisation as A3, then verify os.path.getsize(path) > 0 before reporting success.

#### A5 — The over-collect cap ends collection permanently · MEDIUM · silent under-delivery

collect\_cap (core/scrapper.py:593-595) bounds candidates, the loop breaks on it at :667-668, and **collection is never re-entered after the detail pass**. Every full\_pass rejection at :720 therefore costs a lead the feed could have replaced.

The trigger is a review-count filter rather than require\_email: the feed supplies no review\_count, so :534 pushes every cheap-passing card into pending until the cap is hit.

The user is then told "N of M requested — Google Maps had fewer" (\_export\_summary, ui/screen\_results.py:482-483) — an affirmative claim about market size the code has no basis for.

**Repro:** single domain + area, max\_results = 25, Min reviews > 0 → strict = True, so collect\_cap = 100. All 100 cheap-passing cards go to pending, the loop breaks at :667, the detail pass visits 100 place pages; if 9 clear the threshold the run returns 9 — with the rest of the feed never scrolled, while the same search unfiltered returns far more.

**Fix direction:** wrap collection + detail in an outer loop; after the detail pass, if `matched < target` and neither the end-of-list branch ( :679–681 ) nor the stall branch ( :682–684 ) fired, raise `collect_cap` and keep scrolling from the existing `seen` set. Replace the `completed` boolean ( :730 ) with a reason ( `target_met` / `feed_exhausted` / `budget_exhausted` ), plumb it through `domain_finished_signal`, and have `_export_summary` say "stopped after N candidates checked" for the budget case.

## A6 — A real error is overwritten by "Done — no results" · MEDIUM · silent failure

`run()`'s `finally` ( `core/scrapper.py:869–876` ) emits `done_signal` unconditionally, after the `except` at :866–867 has emitted `error_signal`, and nothing in `ResultsScreen` suppresses `on_done` after an error. Both are queued to the GUI thread in order, so the status label shows `Error: ...` for one turn and is then replaced by "Done — no results" ( `ui/screen_results.py:555` ) plus a toast reading "no results found. Skipping CSV." ( :479 ) — or "Done — all searches scraped" ( :557 ) in multi mode.

This covers the most common first-run failure of all: `uc.Chrome()` raising at :94 on a Chrome/chromedriver mismatch that `_chrome_major_version` ( :50–73 ) can only best-effort mitigate. Nothing indicates Chrome never launched — and the 60-character truncation at :565 would have hidden most of the chromedriver message anyway.

**Fix direction:** decide the terminal state in one place — track the failure in `run()` and either skip `done_signal` or widen it to `done_signal(ok: bool, message: str)`. Minimum UI-side fix: set `self._errored = True` in `on_error` and early-return from `on_done`, showing the full exception in a persistent toast.

## A7 — Worker exception in a multi-task run exports nothing and claims success · MEDIUM

`scrape_domain_progressive` has no local `try`, so an unhandled Selenium error propagates out of the task loop and aborts **every remaining task**. In multi mode only `_on_domain_finished` writes a CSV, and it is never reached for the in-flight task — so that task gets no CSV, the remaining tasks are skipped, and `on_done` ( :556–557 ) reports "Done — all searches scraped".

The rows are not destroyed here: they stay in `self.results` and the table, Export CSV remains enabled ( `_set_idle_mode`, :220–228 ), and `_current_task()` ( :468–474 ) produces correct labels. But nothing tells the user to export, and the success message actively discourages it.

**Repro:** 3 domains × `Toronto`; during task 2, with ~30 rows in the table, close the Chrome window the scraper is driving.

**Fix direction:** set a failure flag in `on_error` that `on_done` cannot overwrite; export `self.results` for `_current_task()` before going idle; report which ( `domain`, `area` ) pairs were never attempted.

## A8 — Closing the window during enrichment loses the partial CSV · MEDIUM

`MainWindow.shutdown_worker()` ( `ui/app.py:489–508` ) escalates stop → abort → terminate on a fixed 10-second budget. **Neither `stop()` nor `abort()` can interrupt an in-flight `enrich_website()` call** ( `core/enrich.py:148–179` — plain `urllib`, two sequential 8-second-timeout fetches, unbounded DNS) reached from `core/scrapper.py:658` or :719: `abort()` force-quits the *driver*, which an `urlopen` is not waiting on.

If that single call outlives both `wait(5000)` calls, `worker.terminate()` ( `ui/app.py:507` ) kills the thread, `run()`'s `finally` ( `core/scrapper.py:869–876` ) never runs, `done_signal` never fires, and `on_done` — the only code that writes the partial CSV ( `ui/screen_results.py:540–548` ) — never executes. The rows are lost **despite** `shutdown_worker` deliberately arming that path with `_stopped_by_user = True` ( `ui/app.py:500` ). `terminate()` also skips the `finally`'s `driver.quit()`.

Confirmed with an offscreen PyQt5 harness: a cooperative exit yields `['finally-ran', 'on_done-slot-ran']`; the terminate branch yields `['terminated']` only.

**Fix direction:** don't depend on a signal `terminate()` prevents — export directly from `shutdown_worker` behind a one-shot guard. Independently, make the cooperative stop land inside the 5-second budget by checking `_running` between the home-page fetch ( `core/enrich.py:160` ) and the contact-page fetch ( :171 ), and lowering the per-fetch timeout.

## A9 — A malformed link on one website kills the whole run · MEDIUM · reproduced

`enrich_website` calls `extract_contacts` (`core/enrich.py:164`) and `_guess_contact_url` (`:168`) **outside** the `try/except` that begins at `:170`. `_guess_contact_url` pipes an unvalidated third-party href into `urllib.parse.urljoin` (`:185`), which raises `ValueError("Invalid IPv6 URL")` whenever an unmatched `[` or `]` lands in the URL's authority component.

### Reproduced:

```
'http://[malformed/contact'    -> RAISES ValueError: Invalid IPv6 URL
'/contact'                    -> 'https://biz.com/contact'    (fine)
'javascript:void(0)/*contact*/' -> 'javascript:void(0)/*contact*/'  (see below)
'file:///C:/Users/contact.html' -> 'file:///C:/Users/contact.html'  (see below)
```

Brackets in a path or query are harmless, so the trigger is narrow — but it falsifies the *"Never raises"* contract at `core/enrich.py:151`, escapes `do_enrich` (`core/scraper.py:612–619`, no handler) at `:658 / :719`, exits `scrape_domain_progressive`, and lands in the single catch-all at `:866–867`. One malformed link on one business website therefore ends a multi-city job with a bare `Invalid IPv6 URL` and abandons every remaining `(domain, area)`. The same applies to `urlparse` at `:101` on a bracketed base host.

Separately, `_fetch` (`:59–74`) never checks the scheme and the `https://` normalisation at `:155–156` covers only the entry URL — so a `file://`, `ftp://` or private-IP `http://` href that matches the `contact|about|reach|connect` regex is fetched and mined for emails, meaning **local file contents can land in the CSV**.

**Fix direction:** move `:164` and `:168` inside the `try` (or catch `ValueError` inside `_guess_contact_url` and return `""`), and in `_fetch` reject any URL whose scheme is not `http / https` before calling `urlopen`. Optionally require the contact URL's host to match the site host.

## A10 — `_guess_contact_url` picks "About" over "Contact" · MEDIUM · missed data

`_guess_contact_url` (`core/enrich.py:182–186`) returns the **first** href in document order matching `contact|about|reach|connect`, scanning every href in the document — `<link>` tags in `<head>`, fragment-only anchors and non-HTTP schemes included — with no scoring and no retry, and `enrich_website` tries only that one candidate (`:167–177`).

The dominant case is ordinary navigation: *About* precedes *Contact* in almost every site's nav, so `/about-us/` wins over `/contact/` — the page least likely to publish an address. A WordPress/Wix `<head>` carrying `<link rel="dns-prefetch" href="//connect.facebook.net">` beats the nav entirely (*connect* matches). A one-pager's `#contact` resolves back to the homepage that just failed.

Because the contact page is the *only* mechanism for recovering an email absent from the homepage, the record ships with an empty email — or is dropped outright when `require_email` is set (`core/filters.py:117–118`) — with nothing logged.

**Fix direction:** collect candidates instead of returning at `:185`; skip `<link>/rel` elements, fragment-only and non-http(s) schemes, and off-host hosts; score `contact` above `about / reach / connect`; attempt the top one or two in rank order.

## A11 — The parser test verifies nothing on a fresh clone · MEDIUM · reproduced

`.gitignore:13` ignores `debug/`, and the only fixture — `debug/initial_Roofing_company.html`, referenced at `tests/test_parse.py:14–18` — is **untracked**.

### Reproduced:

```
$ git ls-files debug/
(empty)
```

`tests/test_parse.py:37-39` turns the missing file into a `return` rather than a failure, after which `:80` prints **"ALL TESTS PASSED"** and the process exits 0. `README.md:405` presents that test as asserting *"every business's name, rating, phone, website, coordinates and Place ID come out correctly, and that sponsored cards are flagged."* On any clone, none of that runs.

The fixture also cannot be regenerated by the app: `_debug_dump` (`core/scrapper.py:512`) is only ever called with `blocked_ / nofeed_ / noresults_` tags (`:574`, `:582`, `:693`) — nothing writes an `initial_*` snapshot. This is also what lets A1 hide.

**Fix direction:** track the fixture — move it to `tests/fixtures/feed_roofing.html` (outside the ignored `debug/` tree) and update `:14-18`, or add `!debug/initial_*.html` after `.gitignore:13`. Independently make `:37-39` `raise SystemExit(1)` so `:80` cannot print without the assertions having run.

## A12 — Phone fallback truncates the leading ( and can become the category · LOW · reproduced

`_PHONE_RE` (`core/parse.py:33`) is `(?:\+?\d{1,3}[\d-]{0,4}){6,}\d{3,4}` — it must start with an optional `+` followed by a **digit**, so a North-American formatted number loses its opening parenthesis. And when the phone row is the only info row available, `_extract_category_address` returns it as the category.

**Reproduced** with a minimal card whose only leaf row is `(416) 469-1939`:

```
phone      = '416) 469-1939'    <- leading '(' lost
category   = '(416) 469-1939'   <- phone surfaced as the category
address    = ''
```

Impact is bounded: `span.UsdLK` is the primary phone source (`:166`) and yields the clean value, so the fallback only fires on cards where that span is absent. But when it fires, the CSV carries an unbalanced-parenthesis phone number that breaks downstream dialers and dedup.

**Fix direction:** allow an optional leading `(` in `_PHONE_RE`, and reject a candidate category that `_PHONE_RE` matches.

## A13 — Image filenames exported as business emails · LOW · reproduced

`_EMAIL_ASSET_RE` (`core/enrich.py:37`) covers `png|jpe?g|gif|webp|svg|css|js|ico|woff2?` — and nothing else. A retina-suffixed asset filename is a syntactically valid email address, so any modern format slips through.

**Reproduced:**

```
logo@2x.png    -> ''           (blocked)
logo@2x.avif   -> 'logo@2x.avif' (exported as the email)
icon@2x.ttf    -> 'icon@2x.ttf'
brochure@2x.pdf -> 'brochure@2x.pdf'
hero@3x.heic   -> 'hero@3x.heic'

extract_contacts('', 'https://biz.com')['email'] -> 'hero@2x.avif'
```

`.avif` is now standard in WordPress/Shopify image pipelines, so this is a live case, not a hypothetical. Note `tests/test_enrich_filters.py:24` pins only the `.png` variant.

**Fix direction:** extend the extension list (`avif|heic|heif|ttf|otf|eot|pdf|mp4|webm|zip`), or invert the rule — require the TLD to be a plausible one rather than blocklisting extensions.

## A14 — Repeated searches silently overwrite the previous CSV · LOW · reproduced

`export_csv` derives a deterministic name, `<domain>_in_<area>.csv` (`core/exporter.py:42-45`), and opens it with mode `"w"` (`:49`). Re-running the same search — or hitting the same `(domain, area)` twice in one matrix — replaces the earlier file with no warning and no backup.

**Reproduced:**



```
export_csv([{"name": "A"}], "car dealers", "Lahore", [{"name"}], tmp) -> car_dealers_in_lahore.csv
export_csv([{"name": "B"}], "car dealers", "Lahore", [{"name"}], tmp) -> car_dealers_in_lahore.csv
file now contains: B          # run 1's rows are gone
```

The adversarial reviewer argued this is defensible product behaviour — a stable, predictable filename per search — and that is a fair reading. It is recorded because it is *silent*: the toast says "CSV saved to ..." either way, and the mid-run Export CSV button writes to the same path the end-of-task export will later overwrite.

**Fix direction (if wanted):** append a timestamp or `-2/-3` suffix when the target exists, or make it a settings toggle.

## A15 — Recent Searches cannot round-trip a multi-area search · LOW

`_load_saved_search`'s areas branch is dead code:

```
self._extra_areas = list(entry.get("areas", []) or [])[1:] if entry.get("areas") else [] # :555
```

`add_saved_search` persists only `{domains, area, max_results}` (`core/settings.py:49-54`) from `areas[0]` (`ui/screen_input.py:691-692`), so `entry.get("areas")` is never truthy. Clicking a Recent Search therefore always clears `_extra_areas`, and `_get_areas()` (`:606-613`) yields one area — running 1 city instead of N.

The reset is visible (the "+N in list" label clears, the button reverts to "List", `:588-595`) and no collected data is lost. The defect is the asymmetry: domains are persisted and restored in full (`:549-552`) while extra areas are never persisted at all.

**Fix direction:** store an `areas` list alongside the existing `area` string (keeping `area` for the entries already on disk) and pass `areas` — not `areas[0]` — from `_on_start` (`:692`). The restore branch then becomes live.

## A16 — The frozen EXE writes `debug/` relative to the working directory · LOW

`_debug_dump` uses CWD-relative paths — `os.makedirs("debug")` (`core/scrapper.py:514`), `debug/{safe}.png` (`:516`), `debug/{safe}.html` (`:517`) — and the project has no `sys.frozen` / `sys._MEIPASS` / `os.chdir` handling anywhere.

In the `--onefile --windowed` build (`BUILD_EXE.bat:123, :39`) the snapshots land under whatever working directory the launcher set. If it isn't writable (EXE under Program Files without elevation), `os.makedirs` raises and is swallowed at `:519-520`, where `print()` is a no-op because `--windowed` leaves `sys.stdout` as `None`. Meanwhile the UI still directs the user there — "See debug/ folder." (`:575, :696`), "Saved debug/nofeed\_.png and .html"\* (`:584`) — as does `README.md:422/425/434`.

`core/settings.py:14` already demonstrates the correct per-user absolute-path pattern.

**Fix direction:** derive the dump root once — `~/mapharvest/debug` when `sys.frozen`, else the repo `debug/` — emit that absolute path in the log lines, and surface failures through `log_signal` rather than `print()`.

## A17 — Two divergent build paths · LOW

`main.spec` is never used by any build in this repo, yet `README.md:448` documents it as *the* packaging file. `BUILD_EXE.bat:121-154` runs `-m PyInstaller ... --specpath "build" ... main.py`, so PyInstaller generates its own `build/MapHarvest-<ver>.spec` each run from the command-line flags — and that generated spec is untracked (`.gitignore:23` ignores `build/`).

The tracked `main.spec` has drifted from the real flags:

| MAIN.SPEC                                         | BUILD_EXE.BAT                                                                                                                                                                           |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>hiddenimports=[]</code> ( <code>:9</code> ) | <code>lxml.etree</code> , <code>lxml._elementpath</code> , plus <code>--collect-all</code> for <code>undetected_chromedriver</code> and <code>selenium</code> ( <code>:130-133</code> ) |
| <code>excludes=[]</code> ( <code>:13</code> )     | <code>tkinter</code> , <code>matplotlib</code> , <code>numpy</code> ( <code>:134-136</code> )                                                                                           |

| MAIN.SPEC                            | BUILD_EXE.BAT                                |
|--------------------------------------|----------------------------------------------|
| name='main' (:25) →<br>dist/main.exe | releases/MapHarvest-<ver>.exe                |
| no version= key                      | --version-file build\version_info.txt (:126) |

Net effect: a maintainer who follows the README and adds a hidden import, data file or exclude to `main.spec` changes **nothing** about the shipped EXE.

**Fix direction:** collapse to one path — either delete `main.spec` and document the batch script, or bring `main.spec` up to date and change the batch file to run `-m PyInstaller main.spec` so a single tracked file governs the build.

A18 — MIT is claimed everywhere but the license text is absent · LOW · reproduced

`tools/gen_version_file.py:14` sets `COPYRIGHT = "MIT Licensed"`, interpolated into `StringStruct('LegalCopyright', ...)`, and `BUILD_EXE.bat:107/ :126` feed the generated resource to PyInstaller — so the string lands in the version resource of **every release build**. `README.md:13` shows a `license-MIT` badge.

**Reproduced:** no `LICENSE/COPYING` file exists on disk or in `git ls-files`. `gen_version_file.py` also stamps `CompanyName/Product Name` as *"MapHarvest"* with no copyright holder, so a recipient of the EXE cannot identify who granted the claimed license.

**Fix direction:** add a `LICENSE` file with the MIT text, a real holder and year; link it from the badge and the legal section; change `COPYRIGHT` to e.g. `"Copyright (c) 2026 <holder> - MIT License"`.

A19 — Dead code and duplicated constants · LOW

| ITEM                                                          | LOCATION                                                                                                           | NOTE                                                                                          |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>T_AFTER_SCROLL = 0.5</code>                             | <code>core/scrapper.py:39</code>                                                                                   | never referenced — growth detection in <code>_scroll_for_more</code> replaced the fixed sleep |
| <code>use_tabs</code> parameter + attribute                   | <code>core/scrapper.py:753, :762</code>                                                                            | explicitly "kept for backward compatibility (unused)"; no caller passes it                    |
| <code>parse_place_url</code> , <code>place_key</code> imports | <code>core/scrapper.py:32</code>                                                                                   | imported but unused since parsing moved wholesale into <code>parse.py</code>                  |
| <code>parse_place_url(href)</code> called twice               | <code>core/parse.py:133-134</code>                                                                                 | the second call only re-reads <code>place_id</code> , which the first already merged          |
| <code>_error</code> key                                       | <code>core/scrapper.py:495</code>                                                                                  | written, never read — see A2                                                                  |
| <code>entry.get("areas")</code> branch                        | <code>ui/screen_input.py:555</code>                                                                                | unreachable — see A15                                                                         |
| <code>main.spec</code>                                        | repo root                                                                                                          | dead and stale — see A17                                                                      |
| Field-class constants duplicated                              | <code>core/scrapper.py:42-46 ↔</code><br><code>ui/screen_input.py:37-41 ↔</code><br><code>core/enrich.py:56</code> | the same detail/enrich field sets in three places; must be edited together                    |

A20 — Documentation drift · LOW

| DRIFT                                                                                                     | LOCATION                      |
|-----------------------------------------------------------------------------------------------------------|-------------------------------|
| Docstring cites <code>tests/test_enrich.py</code> — the file is <code>tests/test_enrich_filters.py</code> | <code>core/enrich.py:5</code> |



| DRIFT                                                                                                                                                                                                                                                      | LOCATION                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| The whole EXE build pipeline ( <code>BUILD_EXE.bat</code> , <code>tools/gen_version_file.py</code> , <code>tools/make_icon.py</code> , <code>app_icon.*</code> , <code>releases/</code> ) is absent from the README, including its project-structure block | <code>README.md:442-472</code>                  |
| <code>main.spec</code> described as " <i>PyInstaller spec (optional packaging)</i> ", implying it is the build path                                                                                                                                        | <code>README.md:448</code>                      |
| The parser test is presented as asserting the full field set, which it cannot do on a clone                                                                                                                                                                | <code>README.md:405</code> (see A11)            |
| Installation says <i>Windows/macOS/Linux</i> , but the build script, <code>_chrome_major_version</code> 's <code>registry/%ProgramFiles%</code> probing and the shipped artifacts are Windows-only                                                         | <code>README.md:333</code>                      |
| <code>debug/</code> is documented as the place to look for failure snapshots without saying where that is in a packaged build                                                                                                                              | <code>README.md:422/425/434</code><br>(see A16) |

Everything else in the README's deep-dive sections still matches the code — the architecture, field list, filter semantics, performance table and troubleshooting are accurate as written.

### Refuted candidates

Raised by a finder, then rejected on adversarial re-reading. Recorded so they don't get re-litigated:

| CANDIDATE                                                                                                              | WHY IT DOES NOT HOLD                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| " <i><code>require_no_website</code> is decided on card data only, so it emits businesses that do have a website</i> " | <code>full_pass</code> re-runs <code>cheap_pass</code> ( <code>core/filters.py:112-113</code> ) after the detail visit and enrichment, so a website discovered later still rejects the record. The remaining gap is only that no visit is <i>forced</i> to look for one — which <code>TODO.md:26</code> already records |
| " <i><code>require_website</code> / <code>require_no_website</code> are decided from feed-card data only</i> "         | Same mechanism as above                                                                                                                                                                                                                                                                                                 |
| " <i>Give-up termination sets the same <code>ended</code> flag as real end-of-list</i> "                               | The flags are conflated, but the only user-visible consequence is the wording already covered by A5                                                                                                                                                                                                                     |
| " <i>Start Scraping does nothing but wobble a read-only field when the persisted export folder no longer exists</i> "  | <code>validate()</code> ( <code>ui/screen_input.py:653</code> ) does shake the field, but the folder path is visible in it and the shake is the app's consistent invalid-input signal; not a defect                                                                                                                     |
| " <i>Export CSV ignores the active search filter and overwrites the auto-export with the unfiltered set</i> "          | The table filter is a view over <code>self.results</code> ; exporting the full set is the documented behaviour, and the overwrite is A14                                                                                                                                                                                |
| " <i>The distutils shim installs unconditionally, replacing a working distutils with a non-package stub</i> "          | <code>_install_shim</code> ( <code>core/distutils_compat.py:76-78</code> ) explicitly checks <code>sys.modules</code> for an existing <code>LooseVersion</code> with a <code>.version</code> attribute and returns early                                                                                                |
| " <i>Recent Searches round-trip drops fields and filters</i> "                                                         | Saved searches are a convenience restore, not a data store; the area half is real and retained as A15                                                                                                                                                                                                                   |

### Extension points

The recipes below are the intended seams. Each lists **every** file that has to change — the codebase duplicates a little knowledge across layers, so partial edits fail silently.

## Add a new exportable field

| STEP | FILE                                  | CHANGE                                                                                                                                      |
|------|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <code>core/parse.py:107-122</code>    | add the key to the <code>data</code> dict init, plus the extraction code                                                                    |
| 2    | <code>core/exporter.py:4-28</code>    | add <code>FIELD_LABELS["your_key"] = "Your Column"</code>                                                                                   |
| 3    | <code>ui/screen_input.py:21-34</code> | append to <b>both</b> <code>FIELD_KEYS</code> and <code>FIELD_NAMES</code> (positionally paired)                                            |
| 4    | <code>core/scrapper.py:42-46</code>   | if it needs a detail visit add it to <code>DETAIL_ONLY_FIELDS</code> ; if a detail visit should refresh it, <code>UPGRADEABLE_FIELDS</code> |
| 5    | <code>core/scrapper.py:449-496</code> | if detail-only, extract it in <code>_extract_detail</code>                                                                                  |
| 6    | <code>ui/screen_input.py:37-41</code> | add to <code>DETAIL_FIELDS</code> / <code>ENRICH_FIELDS</code> so it defaults to off and gets the "slower" tooltip                          |
| 7    | <code>tests/test_parse.py</code>      | assert it against the committed fixture                                                                                                     |

Skipping step 2 is the classic failure: the column appears in the UI, is collected, and is then silently dropped from the CSV by `extrasaction="ignore"`.

## Add a new filter

| STEP | FILE                                                          | CHANGE                                                                                             |
|------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 1    | <code>core/filters.py:40-54</code>                            | add the key to <code>normalize_spec</code> with its coercion                                       |
| 2    | <code>core/filters.py:57-64</code>                            | include it in <code>is_active</code>                                                               |
| 3    | <code>core/filters.py:80-126</code>                           | implement in <code>cheap_pass</code> if card-decidable, else <code>full_pass</code>                |
| 4    | <code>core/filters.py:67-77</code>                            | if it needs late data, add a <code>needs_*</code> function                                         |
| 5    | <code>core/scrapper.py:524-543</code> + <code>:593-595</code> | teach <code>_detail_wants</code> / the <code>strict</code> flag about the new <code>needs_*</code> |
| 6    | <code>ui/screen_input.py:276-396</code>                       | build the widget                                                                                   |
| 7    | <code>ui/screen_input.py:618-631</code>                       | emit it from <code>get_filters()</code>                                                            |
| 8    | <code>ui/screen_input.py:465-474</code>                       | reset it in <code>_reset_filters</code>                                                            |

Missing step 5 is the subtle one: the filter will work but will silently reject records whose data was never fetched.

## Recover from a Google DOM change

1. Reproduce, then grab a fresh dump: the three failure paths already write `debug/*.png` + `.html` (`core/scrapper.py:512-520`). For a working page, add a one-off `_debug_dump(driver, "probe")`.
2. Copy the capture into `debug/` and point a copy of `tests/test_parse.py` at it — the parser is pure, so iterate offline with no browser in the loop.
3. Update the selector in the `selector_registry` table and keep the old one as a fallback; every helper already accepts a comma-separated selector list (`_first_text` / `_first_attr`, `core/scrapper.py:290-311`).
4. Re-run both suites.

## Swap the export format

`core/exporter.py` has one public function and one caller path (`ResultsScreen._notify_task_export`, `screen_results.py:500-511`). Add `export_xlsx(...)` / `export_json(...)` beside `export_csv` with the same `(rows, domain, area, fields, output_path) -> path` signature and switch on a settings key. The records are plain string-valued dicts, so no conversion layer is needed.

## Make enrichment concurrent ( `TODO.md:22` )

The change is contained because `enrich_website` is pure I/O with no shared state. The shape: collect the cards needing enrichment, run them through a `ThreadPoolExecutor(max_workers=4-8)`, then apply results and `full_pass` on the worker thread. Constraints to respect: emit order feeds the live table, `_wait_if_paused()` / `_running` must still be honoured between batches, and per-host politeness (don't fan out 8 requests at one domain).

---